Master's Degree in Computer Science

AUTOMATIC VERIFICATION OF INTELLIGENT SYSTEMS

Course Notes

Academic Year 2026/2027



Prepared by

Matt Clifford Trinidad Ramos

Contents

Reading guide **iv**

1 Course introduction **1**
1.1 Identity, logistics, and assessment . . . 1
1.2 Learning objectives . . . 1

2 An intelligent system: adaptive video compression **2**
2.1 Problem and quality objective . . . 2
2.2 Plant boundary and feedback architecture . . . 2
2.3 A simple plant model . . . 3
2.4 Online least-squares estimation . . . 3
2.5 Integral control and its online realization . . . 4
2.6 Closed-loop state equations and equilibrium . . . 5
2.7 Stability and corrected pole placement . . . 5
2.8 Accounting for the source's intermediate derivation . . . 6
2.9 Adaptive operation and initialization . . . 7
2.10 Key takeaways . . . 8

3 Continuous-time dynamical systems **9**
3.1 State-space representation . . . 9
3.2 Quadrotor: Newton–Euler dynamics . . . 9
3.3 Robotic arm: Euler–Lagrange dynamics . . . 10
3.4 Equilibrium, stability, and controllability . . . 10
3.5 Taxonomy of continuous physical systems . . . 11
3.6 Key takeaways . . . 11

4 Discrete-time dynamical systems **12**
4.1 Difference equations and digital execution . . . 12
4.2 Digital motor control: forward Euler . . . 12
4.3 Exact integration under zero-order hold . . . 13
4.4 Exact and approximate displacement . . . 13
4.5 Leslie population dynamics . . . 14
4.6 PageRank as a Markov chain . . . 14
4.7 Discrete susceptible–infectious–recovered dynamics . . . 15
4.8 Schur stability and exact sampling . . . 15
4.9 Discrete Lyapunov analysis . . . 16
4.10 Comparative taxonomy . . . 16
4.11 Key takeaways . . . 16

5 Optimal control in continuous time **17**
5.1 Problem formulation . . . 17
5.2 Pontryagin's minimum principle . . . 17
5.3 Dynamic programming and the HJB equation . . . 18
5.4 Continuous-time linear quadratic regulator . . . 18
5.5 PMP and HJB: methodological comparison . . . 19
5.6 Key takeaways . . . 19

6 Optimal control in discrete time **20**
6.1 Finite-horizon problem . . . 20
6.2 Discrete adjoint and stationarity conditions . . . 20
6.3 Bellman recursion and feedback . . . 20
6.4 Discrete-time linear quadratic regulator . . . 21
6.5 Continuous and discrete optimal control compared . . . 22
6.6 Key takeaways . . . 22

7 LLM training: supervised fine-tuning and reinforcement learning **23**
7.1 From pretraining to supervised instruction following . . . 23
7.2 Historical examples and the role of data quality . . . 24
7.3 Why introduce rewards? . . . 24
7.4 Classic RLHF: demonstrations, preferences, and policy updates . . . 25
7.5 Direct preference optimization . . . 26
7.6 DeepSeek-R1-Zero and group relative policy optimization . . . 27
7.7 DeepSeek-R1: a hybrid training workflow . . . 28
7.8 Comparing SFT, preference fitting, and online RL . . . 28
7.9 How the stages fit together . . . 29
7.10 Key takeaways and literature route . . . 30

8 Exact dynamic programming: deterministic and stochastic **31**
8.1 Deterministic dynamic programming . . . 31
8.2 The dynamic programming algorithm . . . 33
8.3 Approximation in value space and Q-factors . . . 35
8.4 Stochastic dynamic programming . . . 36
8.5 Deterministic and stochastic DP compared . . . 38
8.6 Key takeaways from the assigned excerpt . . . 38

9 Numerical integration: a simple summary **39**
9.1 Book 2.1: Introduction . . . 39
9.2 Book 2.2: The approximation accuracy . . . 39
9.3 Book 2.3: Euler integration . . . 40
9.4 Book 2.4: The domain of numerical stability . . . 40

10 Quick review **42**
10.1 Core distinctions . . . 42
10.2 Exam revision prompts . . . 42
10.3 Compact terminology . . . 43

A Corrections and source ambiguities **44**

B Source coverage map **46**

C References and compilation **49**
C.1 Primary lecture material . . . 49
C.2 Assigned textbook excerpt . . . 49
C.3 Reference cited by the lecture . . . 49
C.4 Literature cited by the LLM lecture . . . 49
C.5 Supplementary checks . . . 50
C.6 Compiling the source . . . 50

Reading guide

These notes develop the supplied lecture material into a continuous account of modelling, learning, feedback control, and optimal decision making. The source order is preserved: course introduction, adaptive video compression, continuous-time systems, discrete-time systems, continuous-time optimal control, discrete-time optimal control, and LLM training with SFT and reinforcement learning. An assigned excerpt from Bertsekas then develops exact finite-horizon dynamic programming through deterministic and stochastic models, ending at printed page 17 immediately before Section 1.3.

Scope and attribution. The course title and lecturer are taken from the introductory deck. Four technical decks are dated September 27, 2026, and the LLM lecture is dated October 1, 2026; no academic year is explicitly stated. The discrete-time systems deck identifies itself as "Dynamic Systems & Discrete Modeling" and uses "Control Theory" in its footer, without naming an individual author. It is included as supplied course material without adding an author attribution to that deck.

Added lecture and file correspondence. The current optimal-control uploads, `1000-opt-ctr-ct-sys.pdf` and `2000-opt-ctr-dt-sys.pdf`, are identical to the earlier files prefixed `4000` and `5000`. Their content is already developed in Sections 5 and 6 and is counted once. Section 7 adds all 15 pages of `3000-RL4LLM.pdf`. The seven unique lecture decks contain 82 physical source pages. Section 8 additionally covers Dimitri P. Bertsekas, *Reinforcement Learning and Optimal Control*, Chapter 1, printed pp. 2–17 (PDF pp. 15–30), and stops before Section 1.3 exactly as assigned.

Section 9 adds a simple summary of Cellier and Kofman's *Continuous System Simulation*, Sections 2.1–2.4: numerical integration, approximation errors, Euler methods, and numerical stability. It uses printed pp. 25–42 and stops before Section 2.5.

How to read corrections. The mathematical development follows the slides, with explicitly identified corrections where their equations or claims are inconsistent. Explanatory assumptions and derived intermediate steps make the notes usable independently. In particular, the video-compressor slides contain an incorrect characteristic-polynomial expansion. The corrected derivation is given in section 2.7, and the original reasoning is accounted for in section 2.8.

Visual provenance convention. Burgundy *Source correction* boxes mark places where the supplied material is mathematically or technically inconsistent and the correction is derived explicitly. Grey *Technical note* boxes clarify assumptions or interpretation. Light *Supplementary clarification* boxes contain useful context supported by references beyond the supplied lecture decks; they are not presented as additional lecture content.

Source references. Page references denote physical PDF pages, starting at 1. Several slide footers have incorrect totals: the compressor PDF contains 28 pages despite a denominator of 29, the discrete-time systems PDF has 14 pages despite a denominator of 13, and three six-page decks show a denominator of 1. The coverage map in Appendix B uses the actual pages.

Notation conventions

Continuous time is written as t; discrete time is normally k. The compressor lecture retains its original discrete index t. The symbols x, u, and y denote state, input, and output; their dimensions are specified locally. The symbol λ denotes an eigenvalue in stability analysis and a costate in optimal control. These are different uses. Superscript T denotes transpose, and $Q \succeq 0$ ($Q \succ 0$) means symmetric positive semidefinite (positive definite).

Study route. First identify the state and the timing of the input. Next distinguish the model from the measured system. Then ask whether the objective is tracking, stability, or cost minimization. Finally check the assumptions under which the claimed guarantee holds. An accurate model, a stable model, and a stable adaptive implementation are separate assertions.

1 Course introduction

Source: 1000-intro.pdf, pp. 1–7.

1.1 Identity, logistics, and assessment

The course is *Automatic Verification of Intelligent Systems*, taught by Enrico Tronci in the Computer Science Department at Sapienza Università di Roma. The introductory slide gives the departmental address as Via Salaria 113, 00198 Roma, Italy, and the research website https://raise.uniroma1.it/. Contact: tronci@di.uniroma1.it.

The timetable below reproduces the supplied introduction; it is a record of the slides, not an independently verified current timetable.

Day	Time	Location
Monday	17:00–18:30	Room G50, RM115, Building G, E01P03L001; Viale Regina Elena 295, Roma.
Thursday	11:00–13:30	Same room and address.

Course material and recordings are provided through https://elearning.uniroma1.it/. The list includes the syllabus, slides, books, software, videos, and old exams. The registration instruction explicitly names the *Software Verification* course on that platform. This wording is preserved as a source ambiguity: it differs from the title of this course.

Assessment stated in the introduction

The written examination contributes 50% and the software project contributes 50%. A mark of at least 18/30 is required in *each* component; a passing weighted average alone is insufficient.

1.2 Learning objectives

The first objective is to develop modelling ability for the formalization of requirements, design, and Verification and Validation (V&V) of intelligent systems. A model should make the relevant assumptions, behaviours, and desired properties explicit enough for analysis.

The second objective is to understand the mathematical tools that enable the design and analysis of these systems. The third is to know software tools that support automatic design and analysis. The fourth is research-oriented: acquiring the knowledge needed to design algorithms that advance the state of the art in automated design and analysis of intelligent systems.

The following case study connects these objectives: a software controller learns an environment model online and adapts its decisions using a mathematically analysed feedback law. The final introductory slide is a closing acknowledgement and introduces no additional academic content.

2 An intelligent system: adaptive video compression

Source: 1500-video-compressor.pdf, pp. 1–28.

2.1 Problem and quality objective

The case study concerns sending video frames, such as JPEG images, over the Internet. Compression reduces bandwidth demand and can speed up transmission, but excessive compression damages image quality. The design objective is to adjust compression automatically while maintaining an assigned quality level.

The lecture cites Antonio Filieri, Henry Hoffmann, and Martina Maggio, *Automated design of self-adaptive software with control-theoretical formal guarantees*, ICSE 2014, ACM, pp. 299–310, DOI: 10.1145/2568225.2568272. The compressor deck's title page gives the research website http://mclab.di.uniroma1.it/ and the same lecturer, department, address, and email as the introduction.

Quality is measured using the *Structural Similarity Index* (SSIM), by comparing the original frame with its compressed version. The slides use a quality scale from 0 (worst) to 1 (best), with 0.9 as an example reference. This is the operating interpretation used in the case study.

Supplementary clarification: SSIM range

General SSIM formulations can yield negative values; the interval $[0, 1]$ should not be treated as a universal mathematical range. This qualification is supported by the original SSIM paper listed in Appendix C.

The relationship between the compressor setting and quality is not known in advance. Some images compress easily with little perceptual degradation; others do not. Consequently, a controller designed for one movie may not behave identically on another.

2.2 Plant boundary and feedback architecture

The *plant* is the system being controlled. In the first architecture, it contains the movie/frame generator, compressor, and SSIM evaluator. The controller receives the desired quality and feedback about the quality actually attained. Its output is the compression setting.

The revised abstraction in source pp. 5–6 replaces these internals by a single plant block. The same reasoning can then be used for any reasonable plant within the modelling assumptions: learn the model online and adjust the control strategy. In the lecture's terminology, online control is a form of online planning. Combining learning with planning is the distinguishing feature of the intelligent system in this example.

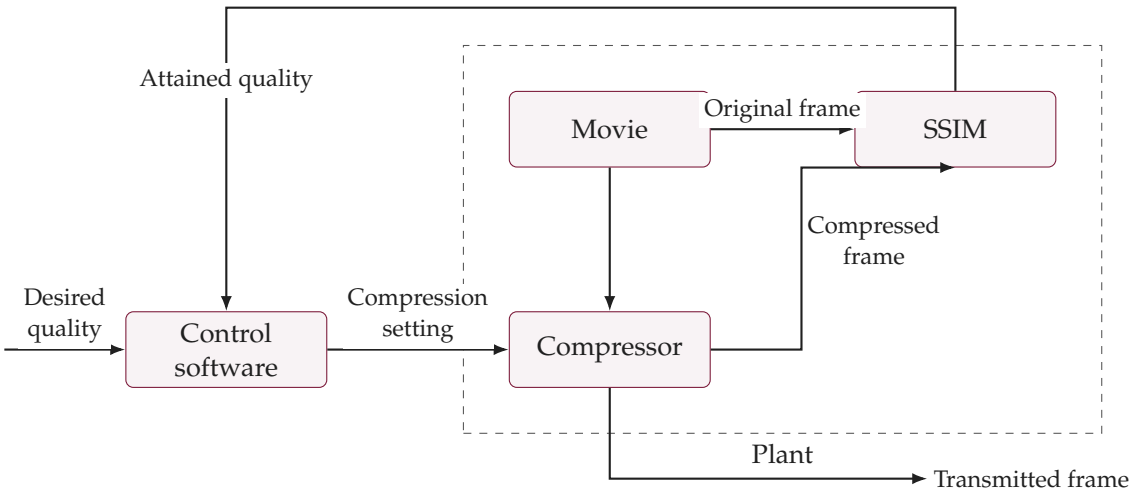


Figure 2.1: Detailed plant boundary from source p. 4. Both original and compressed frames reach SSIM; attained quality returns to the controller.

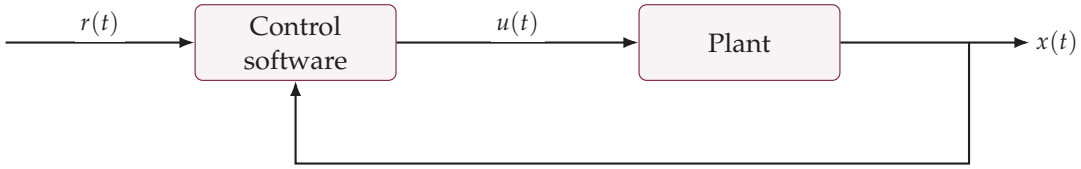


Figure 2.2: Abstract quality-feedback loop. The time-index convention is causal: $u(t)$ affects the next frame's quality $x(t+1)$.

2.3 A simple plant model

Let $x(t)$ denote the quality of the frame sent at discrete time t, and let $u(t)$ be the setting used for the next frame. The proposed model is

$$x(t+1) = \alpha u(t). \tag{2.1}$$

The scalar α is an unknown plant gain. The model has a one-step input-to-output delay and no explicit dependence on the previous quality. It is a deliberately simple local approximation, not a claim that all video content obeys a globally linear law.

The reference $r(t)$ specifies desired quality; for example $r(t) \equiv 0.9$. The controller should make $x(t)$ close to $r(t)$. Choosing a model structure turns the broad task of learning a plant into the narrower task of estimating α.

Clarification. The slides call u a "compression rate", but do not specify the codec parameterization. With $\alpha > 0$, increasing u increases predicted quality. Thus u must be interpreted as a setting with that orientation; it must not automatically be identified with a ratio for which larger values mean more destructive compression. The codec limits and feasible quality range are unspecified.

2.4 Online least-squares estimation

At time t, use a sliding window of w measured qualities

$$\widehat{x}(t), \widehat{x}(t-1), \ldots, \widehat{x}(t-w+1).$$

The corresponding model predictions are $\alpha u(t-1), \alpha u(t-2), \ldots, \alpha u(t-w)$. Each measurement must be paired with the input that produced it, one time step earlier.

Define the sum of squared errors

$$J(\alpha) = \sum_{k=0}^{w-1} [\widehat{x}(t-k) - x(t-k)]^2 = \sum_{k=0}^{w-1} [\widehat{x}(t-k) - \alpha u(t-k-1)]^2. \tag{2.2}$$

The slides call J the Mean Squared Error (MSE); strictly the mean is J/w. Multiplication by the positive constant $1/w$ does not change the minimizing gain.

Differentiating with respect to α gives the complete sequence of steps used in the source:

$$\begin{aligned}
\frac{dJ}{d\alpha} &= \sum_{k=0}^{w-1} 2[\widehat{x}(t-k) - \alpha u(t-k-1)][-u(t-k-1)] \\
&= 2 \sum_{k=0}^{w-1} [-\widehat{x}(t-k)u(t-k-1) + \alpha u^2(t-k-1)] \\
&= -2 \sum_{k=0}^{w-1} \widehat{x}(t-k)u(t-k-1) + 2\alpha \sum_{k=0}^{w-1} u^2(t-k-1).
\end{aligned}$$

Setting this derivative to zero yields

$$\widehat{\alpha}(t) = \frac{\sum_{k=0}^{w-1} \widehat{x}(t-k)u(t-k-1)}{\sum_{k=0}^{w-1} u^2(t-k-1)}. \tag{2.3}$$

The source writes this estimate as $\alpha(t)$; a hat is added here to distinguish the estimate from the true plant gain. Every term uses known past inputs and observed qualities.

If $S_u = \sum_{k=0}^{w-1} u^2(t-k-1) > 0$, then $d^2J/d\alpha^2 = 2S_u > 0$, so the stationary point is the unique minimizer. If every input in the window is zero, $S_u = 0$: the data do not identify α, and the quotient is undefined. A usable implementation must handle this case and must not divide by a zero estimated gain when computing the controller.

2.5 Integral control and its online realization

The error is desired quality minus attained quality:

$$e(t) = r(t) - x(t). \tag{2.4}$$

For a fixed gain $K > 0$, the proposed law is

$$u(t) = K \sum_{k=0}^{t} e(k). \tag{2.5}$$

Persistent positive error increases the setting, while negative error decreases it. The slides refer to a PI (proportional-integral) controller. As written, equation (2.5) is an integral-only law: there is no separate proportional term $K_P e(t)$ in addition to the sum.

An accumulator avoids recomputing the full history:

$$z(t+1) = z(t) + Ke(t), \qquad z(0) = 0, \tag{2.6}$$

$$u(t) = z(t) + Ke(t). \tag{2.7}$$

Indeed, $z(t) = K\sum_{k=0}^{t-1} e(k)$ for fixed K, so the output equals equation (2.5). The current error is included immediately, and $u(t) = z(t+1)$. Therefore

$$\begin{aligned} u(t+1) &= z(t+1) + Ke(t+1) \\ &= z(t) + Ke(t) + Ke(t+1) = u(t) + Ke(t+1). \end{aligned}$$

2.6 Closed-loop state equations and equilibrium

Combining the plant, accumulator, and error gives

$$\begin{aligned} x(t+1) &= \alpha z(t) + \alpha K e(t) = \alpha[z(t) + K(r(t) - x(t))], \\ z(t+1) &= z(t) + Ke(t) = z(t) + K(r(t) - x(t)). \end{aligned}$$

Expanding the terms produces the two-dimensional system

$$x(t+1) = -\alpha K x(t) + \alpha z(t) + \alpha K r(t), \tag{2.8}$$
$$z(t+1) = -Kx(t) + z(t) + Kr(t). \tag{2.9}$$

For $X(t) = [x(t), z(t)]^T$, the matrix form is

$$X(t+1) = AX(t) + B_r r(t), \quad A = \begin{bmatrix} -\alpha K & \alpha \\ -K & 1 \end{bmatrix}, \quad B_r = \begin{bmatrix} \alpha K \\ K \end{bmatrix}. \tag{2.10}$$

Here the reference is the external input to the closed loop; it must not be confused with the compressor input u.

The design goals are (i) equilibrium tracking and (ii) asymptotic stability. For a constant reference r^*, equilibrium means $x(t+1) = x(t) = x^*$ and $z(t+1) = z(t) = z^*$. From the second equation, $K(r^* - x^*) = 0$, so $K \neq 0$ implies $x^* = r^*$. Substitution gives exactly the source's equilibrium equations:

$$\begin{aligned} r^* &= -\alpha K r^* + \alpha z^* + \alpha K r^*, \\ z^* &= -Kr^* + z^* + Kr^*. \end{aligned}$$

They reduce to $r^* = \alpha z^*$ and the identity $z^* = z^*$; hence

$$x^* = r^*, \qquad z^* = \frac{r^*}{\alpha}, \qquad u^* = \frac{r^*}{\alpha}. \tag{2.11}$$

This requires $\alpha \neq 0$ and a feasible actuator setting. The equilibrium location does not depend on K, but convergence to it does. A constant-reference equilibrium does not establish exact tracking of arbitrary time-varying references.

2.7 Stability and corrected pole placement

For a discrete-time linear system, all eigenvalues of A must have modulus strictly less than one for asymptotic stability. Direct expansion gives

$$\begin{aligned} \det(\lambda I - A) &= \det \begin{bmatrix} \lambda + \alpha K & -\alpha \\ K & \lambda - 1 \end{bmatrix} \\ &= (\lambda + \alpha K)(\lambda - 1) + \alpha K \\ &= \lambda^2 + (\alpha K - 1)\lambda = \lambda[\lambda - (1 - \alpha K)]. \end{aligned} \tag{2.12}$$

Thus the two eigenvalues are

$$\lambda_1 = 0, \qquad \lambda_2 = 1 - \alpha K. \tag{2.13}$$

The constant terms cancel; equivalently $\det A = 0$. Therefore

$$|1 - \alpha K| < 1 \quad \Longleftrightarrow \quad 0 < \alpha K < 2. \tag{2.14}$$

For the intended $K > 0$, this requires a positive plant gain and $0 < K < 2/\alpha$.

Correct gain for a chosen nonzero pole

For a known positive α, choose $p \in (0, 1)$ and set

$$K = \frac{1 - p}{\alpha}.$$

The closed-loop eigenvalues are then 0 and p. This is the final gain formula in the slides, but its correct justification is equation (2.12), not the intermediate two-pole argument in source pp. 20–24.

Substitution yields the source's final expanded and error-based forms:

$$x(t+1) = -(1-p)x(t) + \alpha z(t) + (1-p)r(t), \tag{2.15}$$

$$z(t+1) = -\frac{1-p}{\alpha}x(t) + z(t) + \frac{1-p}{\alpha}r(t), \tag{2.16}$$

$$x(t+1) = \alpha z(t) + (1-p)e(t), \tag{2.17}$$

$$z(t+1) = z(t) + \frac{1-p}{\alpha}e(t). \tag{2.18}$$

For a constant reference and a consistent plant history, $x(t) = \alpha z(t)$ after the first plant step. Consequently $x(t+1) = px(t) + (1-p)r^*$, and the tracking error contracts by p. Smaller positive p gives faster nominal convergence; implementation constraints and model error are not captured by this observation.

2.8 Accounting for the source's intermediate derivation

Source correction: characteristic polynomial on source p. 20

The source first gives the correct determinant expression, but expands it as

$$\lambda^2 + (\alpha K - 1)\lambda + \alpha K = 0.$$

For the stated matrix A, the constant terms cancel. The correct characteristic polynomial is

$$\lambda^2 + (\alpha K - 1)\lambda = \lambda\big[\lambda - (1 - \alpha K)\big],$$

so the poles are 0 and $1 - \alpha K$.

Pages 21–23 use the source polynomial to propose poles p_1, p_2. Reconstructing that argument

with the missing factor of λ in the source's factor expansion restored gives

$$\begin{aligned}(\lambda - p_1)(\lambda - p_2) &= \lambda^2 - (p_1 + p_2)\lambda + p_1 p_2,\\ 1 - \alpha K &= p_1 + p_2, \qquad \alpha K = p_1 p_2,\\ 1 - p_1 p_2 &= p_1 + p_2,\\ 1 - p_1 &= p_2(1 + p_1), \qquad p_2 = \frac{1 - p_1}{1 + p_1},\\ K &= \frac{p_1 p_2}{\alpha} = \frac{p_1}{\alpha}\frac{1 - p_1}{1 + p_1}.\end{aligned}$$

These coefficient relations concern the erroneous polynomial, not the actual closed-loop matrix. For the actual matrix, coefficient matching instead gives $p_1 + p_2 = 1 - \alpha K$ and $p_1 p_2 = 0$: both poles cannot be freely assigned away from zero.

Page 24 also argues from $p_1(1 - p_1)/(1 + p_1) \in (0, 1)$ to an arbitrary substitution $1 - p$ with $p \in (0, 1)$. Membership in an interval does not establish that the expression attains every value in that interval. In fact its maximum on $(0, 1)$ is $3 - 2\sqrt{2}$, reached at $p_1 = \sqrt{2} - 1$. The final formula $K = (1 - p)/\alpha$ is nevertheless valid by the independent corrected calculation above.

The intended design insight is retained: adjusting K inversely with the plant gain keeps the *nominal* poles fixed. If an estimate differs from the true gain, that cancellation is only approximate.

2.9 Adaptive operation and initialization

The source's summary combines learning with control. In notation that distinguishes estimate from truth, its update is

$$\widehat{\alpha}(t) = \frac{\sum_{k=0}^{w-1} \widehat{x}(t-k)u(t-k-1)}{\sum_{k=0}^{w-1} u^2(t-k-1)}, \tag{2.19}$$

$$K(t) = \frac{1 - p}{\widehat{\alpha}(t)}, \quad 0 < p < 1, \tag{2.20}$$

$$\widetilde{x}(t+1) = \widehat{\alpha}(t)u(t), \tag{2.21}$$

$$z(t+1) = z(t) + K(t)[r(t) - x(t)], \tag{2.22}$$

$$u(t) = z(t) + K(t)[r(t) - x(t)]. \tag{2.23}$$

The tilde marks a model prediction: actual measured quality need not equal it. The estimator uses $\widehat{x}$ for measurements in the source; the feedback equations use x for attained quality. In practice the feedback signal is the available measurement.

The summary states $z(0) = 0$, $x(0) = \cdots = x(w-1) = 1$, and control starting at $t = w$. This describes an initial full-quality phase, but does not specify its input settings, the measurement $x(w)$, or how the accumulator is handled before control begins. Those are implementation details that must be supplied; measured values must not be fabricated. At $t = w$, the displayed estimator needs the pairs $(\widehat{x}(1), u(0)), \ldots, (\widehat{x}(w), u(w-1))$.

With variable gains the accumulator contains a weighted sum of past errors. In general it is not $K(t)\sum e(k)$, because each error was accumulated with the gain used at its own time.

For a fixed true gain α and a frozen estimate $\widehat{\alpha}$, the actual nonzero pole is

$$1 - \alpha\frac{1 - p}{\widehat{\alpha}},$$

so nominal pole placement alone does not prove stability under estimation error. Nor does checking every frozen matrix establish stability of an arbitrary time-varying adaptive system. Actuator limits, noise, changes in movie content, and estimation safeguards require further analysis not present in these slides.

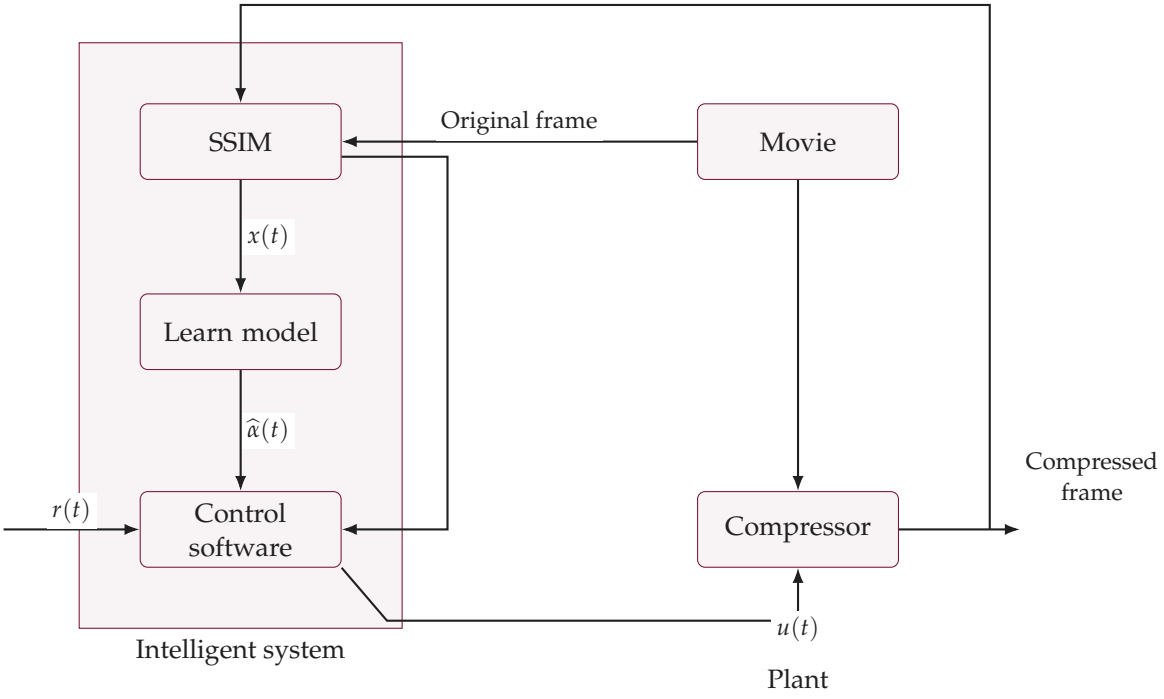


Figure 2.3: Revised boundary in source p. 27: SSIM, model learning, and control belong to the intelligent system; movie and compressor form the plant. Past settings u are also required by the estimator, although that connection is omitted in the source diagram.

2.10 Key takeaways

The case study combines online parameter estimation, integral feedback, and nominal pole placement. Preserve the one-frame delay in every data pair. The least-squares denominator must be nonzero, the gain used in a division must be valid, and the requested equilibrium must be feasible. For the stated fixed-gain model, the poles are 0 and $1 - \alpha K$; with exact model knowledge, $K = (1 - p)/\alpha$ places the nonzero pole at p.

3 Continuous-time dynamical systems

Source: 2000-ct-systems.pdf, pp. 1–6; September 27, 2026.

The source module is titled *Introduction to Continuous-Time Dynamical Systems*, with subtitle *Fundamentals, State-Space Representation, and Robotics Applications*.

3.1 State-space representation

Continuous-time models describe physical processes at times $t \in \mathbb{R}_{\geq 0}$. Their general nonlinear form is

$$\dot{x}(t) = \frac{dx(t)}{dt} = f(t, x(t), u(t)), \qquad y(t) = h(t, x(t), u(t)). \tag{3.1}$$

The state $x(t) \in \mathbb{R}^n$ records the variables needed, together with future inputs, to determine future behaviour. In physical examples, these encode memory and stored energy. The input $u(t) \in \mathbb{R}^m$ represents controls, forces, or disturbances; $y(t)$ is the output. The slides call the state a minimum set of variables. Minimality avoids redundancy, but a valid state representation need not be minimal.

Linear time-invariant special case

A Linear Time-Invariant (LTI) model has constant matrices:

$$\dot{x} = Ax + Bu, \qquad y = Cx + Du.$$

Here $A \in \mathbb{R}^{n \times n}$ is the state matrix and $B \in \mathbb{R}^{n \times m}$ the input matrix. For $y \in \mathbb{R}^q$, the dimensions are $C \in \mathbb{R}^{q \times n}$ and $D \in \mathbb{R}^{q \times m}$. The term Du is direct input-to-output dependence.

3.2 Quadrotor: Newton–Euler dynamics

A quadrotor has six physical Degrees of Freedom (DOFs), three translational and three rotational. Including rates gives twelve state components:

$$x = [\mathbf{p}^T, \mathbf{v}^T, \Theta^T, \boldsymbol{\omega}^T]^T \in \mathbb{R}^{12}.$$

Position is $\mathbf{p} = [x, y, z]^T$, velocity is $\mathbf{v} = [\dot{x}, \dot{y}, \dot{z}]^T$, Euler angles are $\Theta = [\phi, \theta, \psi]^T$, and body angular velocity is $\boldsymbol{\omega} = [p, q, r]^T$. The position vector $\mathbf{p}$ is distinct from the scalar angular-rate component p.

The four inputs are collective thrust F_T and torques $\boldsymbol{\tau} = [\tau_x, \tau_y, \tau_z]^T$. The source equations are

$$\dot{\mathbf{p}} = \mathbf{v}, \tag{3.2}$$

$$\dot{\mathbf{v}} = -g\mathbf{e}_3 + \frac{1}{m} R(\Theta) \begin{bmatrix} 0 \\ 0 \\ F_T \end{bmatrix}, \tag{3.3}$$

$$\dot{\Theta} = W(\Theta)\boldsymbol{\omega}, \tag{3.4}$$

$$I\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (I\boldsymbol{\omega}). \tag{3.5}$$

Here g is gravitational acceleration, m is mass, $\mathbf{e}_3 = [0, 0, 1]^T$, R rotates body thrust into inertial coordinates, W converts body angular velocity into Euler-angle rates, and I is the inertia matrix. The cross product represents gyroscopic coupling. The gravity sign follows the source's vertical-axis convention.

The model is nonlinear and coupled: changing orientation changes thrust direction. Four inputs control six configuration DOFs, so the system is underactuated. The source does not specify the Euler-angle convention or the explicit matrix W; these must be fixed before implementation.

3.3 Robotic arm: Euler–Lagrange dynamics

An n-joint manipulator has joint positions $q \in \mathbb{R}^n$, joint velocities $\dot{q}$, state $x = [q^T, \dot{q}^T]^T \in \mathbb{R}^{2n}$, and torque input $u = \tau \in \mathbb{R}^n$. Its dynamics are

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau. \tag{3.6}$$

The configuration-dependent inertia is $M(q)$, the Coriolis/centripetal term is $C(q,\dot{q})\dot{q}$, and the gravitational torque is $g(q)$. This vector-valued $g(q)$ differs from the quadrotor's scalar gravity constant.

Solving for acceleration converts the second-order model to first-order state space:

$$\frac{d}{dt}\begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ M(q)^{-1}\left(\tau - C(q,\dot{q})\dot{q} - g(q)\right) \end{bmatrix}. \tag{3.7}$$

In standard independent-joint rigid-body coordinates, the inertia matrix is positive definite and invertible. First-order conversion exposes the state but does not remove nonlinear coupling.

3.4 Equilibrium, stability, and controllability

An equilibrium x_e under constant input u_e satisfies $f(t, x_e, u_e) = 0$. If f is explicitly time-dependent, the equality must hold at every relevant time.

Lyapunov stability

An equilibrium is stable if starting sufficiently close makes the trajectory remain arbitrarily close for all future time. For every $\varepsilon > 0$, there is a $\delta > 0$ such that

$$\|x(t_0) - x_e\| < \delta \implies \|x(t) - x_e\| < \varepsilon \quad (t \geq t_0).$$

Asymptotic stability additionally requires convergence $x(t) \to x_e$ for nearby initial states. The slide's phrase "remain bounded" is too weak: staying in a large bounded region need not mean staying close.

For an autonomous LTI system $\dot{x} = Ax$, the origin is asymptotically stable precisely when

$$\operatorname{Re}\lambda_i(A) < 0 \qquad (i = 1, \ldots, n). \tag{3.8}$$

Eigenvalues on the imaginary axis do not meet this strict condition and require separate analysis for marginal stability.

Controllability is the ability to steer $x(t_0)$ to a target $x(t_f)$ using admissible inputs. The slides say bounded inputs. Inputs bounded on a finite interval must be distinguished from a prescribed actuator-amplitude bound: a fixed bound can restrict finite-time reachability even for a controllable LTI pair.

3.5 Taxonomy of continuous physical systems

The source comparison is split into two readable tables. Dimensions refer to the chosen models, not to every possible model of each device.

Table 3.1: States and inputs in the continuous-time taxonomy.

System	State	Input
Quadrotor	Position, attitude, rates ($n = 12$).	Motor thrusts / torques ($m = 4$).
Robotic arm	Joint angles and rates ($n = 2k$ for k joints).	Joint torques ($m = k$).
Self-driving car	Coordinates, yaw, speed; source reports $n = 5$.	Steering and acceleration ($m = 2$).
Inverted pendulum	Cart position, angle, and rates ($n = 4$).	Cart force ($m = 1$).

Table 3.2: Actuation and dynamic features in the continuous-time taxonomy.

System	Classification	Features
Quadrotor	Underactuated	Aerodynamic / gyroscopic coupling.
Robotic arm	Fully actuated	Nonlinear inertia and Coriolis forces.
Self-driving car	Non-holonomic	Kinematic constraints and dynamic tire slip.
Inverted pendulum	Underactuated	Unstable upright (zenith) equilibrium.

Clarification. The car row states $n = 5$ without listing all five components; no missing component is invented here. Non-holonomic describes a motion constraint, not an input count. Aerodynamic effects are listed in the taxonomy but no explicit drag term appears in the displayed quadrotor model.

3.6 Key takeaways

Continuous dynamics specify rates of change. Equilibrium, stability, and controllability ask where motion can stop, how perturbations evolve, and what states inputs can reach. They are distinct properties.

4 Discrete-time dynamical systems

Source: 3000-dt-systems.pdf, pp. 1–14; September 27, 2026.

The module is titled *Introduction to Discrete-Time Dynamical Systems*, with subtitle *State-Space Models, Discretization & Student Walkthroughs*. It connects physical dynamics with digital execution and naturally discrete models.

4.1 Difference equations and digital execution

For $k \in \mathbb{N}_0 = \{0, 1, 2, \ldots\}$, the general state and output equations are

$$x_{k+1} = f(k, x_k, u_k), \qquad y_k = h(k, x_k, u_k). \tag{4.1}$$

The LTI form is

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k + Du_k. \tag{4.2}$$

Here $x_k \in \mathbb{R}^n$, $u_k \in \mathbb{R}^m$, $A \in \mathbb{R}^{n\times n}$, and $B \in \mathbb{R}^{n\times m}$. A difference equation specifies the next state rather than its instantaneous derivative.

Physical systems evolve continuously, but microcontrollers update at sampling times $t_k = kT_s$, with sampling period $T_s > 0$. Discretization converts, for example, $\dot{x} = A_c x$ into $x_{k+1} = A_d x_k$. Its accuracy depends on how A_d is obtained.

Knowing the initial state and input sequence gives the recursive predictions

$$\begin{aligned} x_1 &= Ax_0 + Bu_0, \\ x_2 &= Ax_1 + Bu_1 = A^2x_0 + ABu_0 + Bu_1. \end{aligned}$$

The second equation shows how an earlier action continues to affect future states through the transition matrix.

4.2 Digital motor control: forward Euler

A direct-current (DC) motor is modelled using angle θ, angular velocity ω, and voltage input u:

$$\dot{\theta} = \omega, \qquad \dot{\omega} = -\alpha\omega + \beta u. \tag{4.3}$$

Here α is a damping coefficient and β converts voltage into an acceleration contribution. These are motor parameters, unrelated to the compressor's gain.

Forward Euler approximates $\dot{x}(t_k)$ by $(x_{k+1} - x_k)/T_s$. Applying this to both equations gives

$$\begin{aligned} \frac{\theta_{k+1} - \theta_k}{T_s} &= \omega_k && \Longrightarrow\ \theta_{k+1} = \theta_k + T_s\omega_k, \\ \frac{\omega_{k+1} - \omega_k}{T_s} &= -\alpha\omega_k + \beta u_k && \Longrightarrow\ \omega_{k+1} = (1 - \alpha T_s)\omega_k + \beta T_s u_k. \end{aligned}$$

Thus the Euler matrices are

$$\begin{bmatrix} \theta_{k+1} \\ \omega_{k+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & T_s \\ 0 & 1 - \alpha T_s \end{bmatrix}}_{A_E} \begin{bmatrix} \theta_k \\ \omega_k \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ \beta T_s \end{bmatrix}}_{B_E} u_k. \tag{4.4}$$

Voltage u_k first changes ω_{k+1} and only affects angle at step $k + 2$ in this approximation, because the angle update evaluates velocity at the start of the interval.

4.3 Exact integration under zero-order hold

A Zero-Order Hold (ZOH) keeps u constant during each sample interval. Let $\tau \in [0, T_s]$ be local time from t_k, so $u(t_k + \tau) = u_k$. For $\alpha \neq 0$, solving the velocity equation gives

$$\omega(t_k + \tau) = \omega_k e^{-\alpha\tau} + \frac{\beta u_k}{\alpha}(1 - e^{-\alpha\tau}). \tag{4.5}$$

Integrating velocity gives the source's two contributions to displacement:

$$\theta_{k+1} = \theta_k + \underbrace{\int_0^{T_s} \omega_k e^{-\alpha\tau}\, d\tau}_{I_1} + \underbrace{\int_0^{T_s} \frac{\beta u_k}{\alpha}(1 - e^{-\alpha\tau})\, d\tau}_{I_2}, \tag{4.6}$$

$$I_1 = \frac{1 - e^{-\alpha T_s}}{\alpha}\omega_k, \tag{4.7}$$

$$I_2 = \frac{\beta}{\alpha}\left(T_s - \frac{1 - e^{-\alpha T_s}}{\alpha}\right) u_k. \tag{4.8}$$

These integrals and the velocity at $\tau = T_s$ imply the exact ZOH matrices

$$A_Z = \begin{bmatrix} 1 & \dfrac{1 - e^{-\alpha T_s}}{\alpha} \\ 0 & e^{-\alpha T_s} \end{bmatrix}, \quad B_Z = \begin{bmatrix} \dfrac{\beta}{\alpha}\left(T_s - \dfrac{1 - e^{-\alpha T_s}}{\alpha}\right) \\ \dfrac{\beta}{\alpha}(1 - e^{-\alpha T_s}) \end{bmatrix}. \tag{4.9}$$

The full matrices are an explanatory derivation from the source's integrals. In general, for LTI dynamics under ZOH,

$$A_d = e^{A_c T_s}, \qquad B_d = \int_0^{T_s} e^{A_c \tau} B_c\, d\tau.$$

4.4 Exact and approximate displacement

Expanding the exponential gives

$$e^{-\alpha T_s} = 1 - \alpha T_s + \tfrac{1}{2}\alpha^2 T_s^2 + O(T_s^3).$$

The source uses $I_1 \approx T_s\omega_k$ and $I_2 \approx \frac{1}{2}\beta T_s^2 u_k$ to emphasize the direct within-interval input effect:

$$\theta_{k+1} \approx \theta_k + T_s\omega_k + \tfrac{1}{2}\beta T_s^2 u_k.$$

This mixes the first-order velocity term with the leading input term. A consistent second-order approximation must also retain damping:

$$\theta_{k+1} = \theta_k + T_s\omega_k + \tfrac{1}{2}(-\alpha\omega_k + \beta u_k)T_s^2 + O(T_s^3). \tag{4.10}$$

Likewise the slide comparison

$$B_E = \begin{bmatrix} 0 \\ \beta T_s \end{bmatrix}, \qquad B_Z \approx \begin{bmatrix} \frac{1}{2}\beta T_s^2 \\ \beta T_s \end{bmatrix}$$

keeps the leading nonzero term in each component; it is not the exact input matrix. The second-order lower component is $\beta T_s - \frac{1}{2}\alpha\beta T_s^2$.

Source correction: ZOH does not imply constant total acceleration

ZOH holds u_k constant, not $\dot{\omega} = -\alpha\omega + \beta u_k$. For nonzero damping, velocity changes and therefore acceleration changes during the interval. The term $\frac{1}{2}\beta u_k T_s^2$ is the leading input displacement. For $\alpha = 0$ it is exact, with

$$A_d = \begin{bmatrix} 1 & T_s \\ 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} \frac{1}{2}\beta T_s^2 \\ \beta T_s \end{bmatrix}.$$

4.5 Leslie population dynamics

The Leslie model tracks female population counts in m age cohorts across breeding seasons. Its state is $x_k = [n_{1,k}, \ldots, n_{m,k}]^T$, with $n_{i,k}$ the population in age class i. In the three-class example,

$$\begin{bmatrix} n_{1,k+1} \\ n_{2,k+1} \\ n_{3,k+1} \end{bmatrix} = \underbrace{\begin{bmatrix} f_1 & f_2 & f_3 \\ s_1 & 0 & 0 \\ 0 & s_2 & 0 \end{bmatrix}}_{L} \begin{bmatrix} n_{1,k} \\ n_{2,k} \\ n_{3,k} \end{bmatrix}. \tag{4.11}$$

Fecundity $f_i \geq 0$ is the average female offspring per member of class i. The source assumes $s_i \in (0, 1]$, the survival probability from class i to $i + 1$. The first row counts newborns:

$$n_{1,k+1} = f_1 n_{1,k} + f_2 n_{2,k} + f_3 n_{3,k}.$$

The subdiagonal describes aging: $n_{2,k+1} = s_1 n_{1,k}$ and $n_{3,k+1} = s_2 n_{2,k}$.

The source uses the dominant eigenvalue as the long-term growth factor: $\lambda_1 > 1$ indicates expansion, $\lambda_1 < 1$ decline toward extinction, and $\lambda_1 = 1$ a stationary growth factor.

Supplementary clarification: Conditions for asymptotic age-profile convergence

Under the standard condition that L is primitive and the initial population is nonzero and nonnegative, $\lambda_1^{-k} x_k \to c v_1$, with positive dominant eigenvector v_1 and $c > 0$. This is convergence after scaling by the dominant growth factor; the unscaled difference $x_k - c\lambda_1^k v_1$ need not tend to zero in a growing population. Normalized age proportions approach $v_1 / (\mathbf{1}^T v_1)$. Suitable irreducibility and aperiodicity assumptions are needed for this convergence; without them, cyclic counterexamples are possible.

4.6 PageRank as a Markov chain

PageRank models a random surfer moving across N webpages. The column vector $p_k \in \mathbb{R}^N$ contains probabilities $p_{i,k} \geq 0$ with $\sum_i p_{i,k} = 1$. The source defines a row-stochastic matrix by

$$M_{ij} = \mathbb{P}(\text{page } j \text{ at } k+1 \mid \text{page } i \text{ at } k), \quad \sum_j M_{ij} = 1.$$

Consequently,

$$p_{k+1}^T = p_k^T M \quad \Longleftrightarrow \quad p_{k+1} = M^T p_k. \tag{4.12}$$

The transpose matters: rows describe outgoing transitions, whereas p_k is a column.

The damping adjustment is

$$\overline{M} = dM + \frac{1-d}{N}\mathbf{1}_{N\times N}, \qquad d = 0.85, \tag{4.13}$$

where **1** is the all-ones matrix. The surfer follows links with probability 0.85 and teleports uniformly with probability 0.15.

Supplementary clarification: Dangling pages in PageRank

A page with no outgoing links must first receive a valid stochastic row, conventionally a uniform row. A zero row would otherwise give row sum $1 - d$, not 1, after the adjustment. Teleportation also prevents trapping in closed groups of pages and removes periodicity.

For stochastic M and $0 < d < 1$, the adjusted matrix is strictly positive. Power iteration converges to a unique stationary distribution:

$$p_{k+1} = \overline{M}^T p_k, \qquad p^* = \overline{M}^T p^*. \tag{4.14}$$

The stationary eigenvalue is 1. Convergence occurs within the probability simplex, not toward zero in the full vector space, since total probability remains one.

4.7 Discrete susceptible–infectious–recovered dynamics

The Susceptible–Infectious–Recovered (SIR) model uses susceptible S_k, infectious I_k, and recovered R_k populations at day k:

$$S_{k+1} = S_k - \beta S_k I_k, \tag{4.15}$$

$$I_{k+1} = I_k + \beta S_k I_k - \gamma I_k, \tag{4.16}$$

$$R_{k+1} = R_k + \gamma I_k. \tag{4.17}$$

New infections are $\beta S_k I_k$; recoveries are γI_k. The slides call β a daily infection parameter per contact and γ a daily recovery parameter. Units of β depend on whether compartments are counts or fractions: no explicit division by population size occurs in these equations. Their sum conserves $S_k + I_k + R_k$.

For $I_k > 0$, infection grows in a step if and only if $\beta S_k > \gamma$. With $\gamma > 0$, the source defines the initial reproduction quantity

$$\mathcal{R}_0 = \frac{\beta S_0}{\gamma}. \tag{4.18}$$

Thus $\mathcal{R}_0 > 1$ gives initial growth. Subsequent growth depends on S_k; the threshold does not imply indefinite increase. Nonnegative compartments also require admissible step parameters. For example, $0 \leq \gamma \leq 1$ and $0 \leq \beta I_k \leq 1$ prevent transfers exceeding the available compartments.

4.8 Schur stability and exact sampling

For the autonomous LTI recurrence $x_{k+1} = Ax_k$, the origin is asymptotically stable exactly when

$$|\lambda_i(A)| < 1 \qquad (i = 1, \ldots, n). \tag{4.19}$$

Such A is *Schur stable*. In continuous time the criterion is negative real part; in discrete time it is membership in the open unit disk.

For exact sampling, $A_d = e^{A_c T_s}$ maps a continuous eigenvalue s to

$$z = e^{sT_s}, \qquad |z| = e^{\mathrm{Re}(s)T_s}. \tag{4.20}$$

For $T_s > 0$, the left half-plane maps into the unit disk. Euler instead maps s to $1 + sT_s$, and can lose stability for an excessive step size.

4.9 Discrete Lyapunov analysis

The matrix A is Schur stable if and only if for every symmetric $Q \succ 0$ there exists a unique symmetric $P \succ 0$ satisfying

$$A^T P A - P = -Q. \tag{4.21}$$

For $V(x_k) = x_k^T P x_k$ and $x_k \neq 0$,

$$\begin{aligned} \Delta V(x_k) &= V(x_{k+1}) - V(x_k) \\ &= x_k^T (A^T P A - P) x_k = -x_k^T Q x_k < 0. \end{aligned} \tag{4.22}$$

Positive definiteness relates V to state magnitude, so the decreasing quadratic energy establishes convergence to zero. At the origin, $V = 0$ and $\Delta V = 0$; the strict inequalities in the source implicitly exclude that point.

4.10 Comparative taxonomy

Table 4.1: Discrete models and possible controls listed in the source.

Domain	State and evolution	Input / control listed
Digital control	Position and speed ($n = 2$); ZOH $A_d = e^{A_c T_s}$.	Motor voltage ($m = 1$).
Leslie ecology	Cohort counts ($n = m$); nonnegative L.	Culling / harvesting.
PageRank DTMC	Page probabilities ($n = N$); stochastic $\overline{M}^T$.	Damping / teleportation d.
Discrete SIR	Susceptible, infectious, recovered ($n = 3$); nonlinear recurrence.	Vaccination factor.

DTMC means Discrete-Time Markov Chain. In the Leslie row, m denotes cohorts rather than input dimension. Harvesting and vaccination are mentioned as possible interventions, but no such terms appear in the preceding equations.

The source lists $|\lambda(A_d)| < 1$ for digital control, $\lambda_1 \gtrless 1$ for Leslie growth, $\lambda_1 = 1$ for PageRank convergence, and $\mathcal{R}_0 < 1$ for declining infection. These address different properties. In particular, the open-loop motor's angle integrator gives an eigenvalue 1: the full open-loop state is not asymptotically stable at zero. PageRank convergence needs its stochastic assumptions, while the SIR threshold concerns infection rather than global asymptotic stability of all compartments.

4.11 Key takeaways

Discrete models can describe sampled physics or naturally staged processes. Preserve input timing, matrix transposes, and units. Distinguish exact discretization from numerical approximation, and distinguish LTI stability from the growth and convergence properties of other models.

5 Optimal control in continuous time

Source: 1000-opt-ctr-ct-sys.pdf, pp. 1–6; September 27, 2026.

The module subtitle is *Formulation, Pontryagin's Minimum Principle, and HJB Equations*. Modelling specifies how a system evolves; optimal control chooses admissible actions to minimize a stated performance index.

5.1 Problem formulation

Choose a control trajectory $u^*(t)$ over $[t_0, t_f]$ to minimize

$$J(u) = \phi(x(t_f), t_f) + \int_{t_0}^{t_f} L(t, x(t), u(t))\, dt \tag{5.1}$$

subject to

$$\dot{x}(t) = f(t, x(t), u(t)), \quad x(t_0) = x_0, \quad u(t) \in \Omega \subseteq \mathbb{R}^m, \quad x(t_f) \in S_f \subseteq \mathbb{R}^n.$$

The state is $x \in \mathbb{R}^n$. The function L is the running cost (also called the Lagrangian), and ϕ is the terminal cost. The source distinguishes the set $\mathcal{U}$ of admissible input trajectories from the pointwise input set Ω. The final time may be fixed or free in the general formulation.

The terminal set encodes a required destination; the terminal cost expresses a preference among final states. These have different roles. Subsequent simple boundary formulas assume a free terminal state unless indicated otherwise. The finite-horizon HJB and LQR formulas below use a fixed t_f.

5.2 Pontryagin's minimum principle

The *Hamiltonian* combines running cost with the effect of state motion:

$$H(t, x, u, \lambda) = L(t, x, u) + \lambda^T f(t, x, u). \tag{5.2}$$

It maps $[t_0, t_f] \times \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^n$ to $\mathbb{R}$. The vector $\lambda(t) \in \mathbb{R}^n$ is a costate (adjoint), not an eigenvalue. Along an optimal trajectory, it describes the sensitivity associated with the dynamic constraints.

In the normal smooth setting used by the slides, Pontryagin's Minimum Principle (PMP) gives the following necessary conditions, evaluated along (x^*, u^*, λ^*):

$$\dot{x}^*(t) = \nabla_\lambda H = f(t, x^*(t), u^*(t)), \tag{5.3}$$

$$\dot{\lambda}^*(t) = -\nabla_x H = -\nabla_x L - \left(\frac{\partial f}{\partial x}\right)^T \lambda^*(t), \tag{5.4}$$

$$u^*(t) \in \arg\min_{u \in \Omega} H(t, x^*(t), u, \lambda^*(t)), \tag{5.5}$$

$$\lambda^*(t_f) = \nabla_x \phi(x^*(t_f), t_f) \quad \text{if the terminal state is free.} \tag{5.6}$$

Symbols: x is the state, u the control, L the running cost, ϕ the terminal cost, λ the costate, and Ω the admissible pointwise control set.

Set membership allows for multiple minimizers; the slides write equality when selecting one. The state evolves forward from x_0, whereas the costate has a terminal boundary condition. Solving them together typically forms a two-point boundary-value problem.

Necessary conditions are not an automatic optimality certificate

A trajectory satisfying PMP is a candidate; extra assumptions or comparison are needed to establish global optimality. The simplified Hamiltonian assumes the normal case. Terminal constraints modify equation (5.6), state path constraints require additional conditions, and an interior stationarity equation alone does not replace minimization over a constrained control set.

For an unconstrained interior minimizer, differentiability gives $\nabla_u H = 0$. For free terminal time, an additional transversality condition is needed. In the simple free-terminal-state case it is $H(t_f) + \partial\phi/\partial t_f = 0$. This completes the distinction in the source's general formulation; it is not implied by the terminal-costate condition alone.

5.3 Dynamic programming and the HJB equation

For fixed terminal time, the value function is the minimum remaining cost from state x at time t:

$$V(t,x) = \min_{u|_{[t,t_f]}} \left\{ \phi(x(t_f), t_f) + \int_t^{t_f} L(s, x(s), u(s))\, ds \right\}, \quad x(t) = x. \tag{5.7}$$

The minimization is over admissible trajectories and their induced states. In the slides, dependence of ϕ on fixed t_f is suppressed, written simply as $\phi(x(t_f))$.

Bellman's principle says that the remaining part of an optimal policy must itself be optimal for the state reached. If V is differentiable, a small-time expansion leads to the Hamilton–Jacobi–Bellman (HJB) partial differential equation:

$$-\frac{\partial V}{\partial t}(t,x) = \min_{u\in\Omega}\{L(t,x,u) + \nabla_x V(t,x)^T f(t,x,u)\}. \tag{5.8}$$

The terminal condition and a minimizing feedback policy are

$$V(t_f, x) = \phi(x, t_f), \tag{5.9}$$

$$u^*(t,x) \in \arg\min_{u\in\Omega}\{L(t,x,u) + \nabla_x V(t,x)^T f(t,x,u)\}. \tag{5.10}$$

Symbols: $V(t,x)$ is the optimal cost-to-go, $\nabla_x V$ its state gradient, and the minimization is over admissible controls $u \in \Omega$.

Unlike a trajectory prescribed only as a function of time, the policy can respond to the current state.

Under the regularity and admissibility assumptions of a verification theorem, a solution of HJB with the boundary condition and an attained minimizing policy gives a sufficient global-optimality certificate. A formal PDE expression alone is not such a certificate. Nonsmooth value functions require a generalized treatment not developed in these slides. A terminal set constraint also changes the boundary/domain treatment; equation (5.9) as a finite cost everywhere describes the unconstrained terminal-state case.

5.4 Continuous-time linear quadratic regulator

The finite-horizon Linear Quadratic Regulator (LQR) minimizes

$$J(u) = \tfrac{1}{2} x(t_f)^T P_f x(t_f) + \tfrac{1}{2} \int_{t_0}^{t_f} \left(x(t)^T Q x(t) + u(t)^T R u(t) \right) dt \tag{5.11}$$

subject to $\dot{x} = Ax + Bu$. The standard assumptions are symmetric $Q \succeq 0$, $P_f \succeq 0$, and $R \succ 0$, with unconstrained inputs. State penalties discourage deviation from zero; input penalties discourage excessive actuation. The halves simplify derivatives.

The Hamiltonian is

$$H = \tfrac{1}{2}x^T Q x + \tfrac{1}{2}u^T R u + \lambda^T (Ax + Bu).$$

Stationarity yields

$$Ru + B^T\lambda = 0 \quad \Longrightarrow \quad u^*(t) = -R^{-1}B^T\lambda(t). \tag{5.12}$$

Use the quadratic value-function ansatz $V(t,x) = \frac{1}{2}x^T P(t)x$, with symmetric $P(t)$. Then $\lambda = \nabla_x V = P(t)x$ along the optimal trajectory. Substitution into HJB gives the differential Riccati equation

$$-\dot{P}(t) = A^T P(t) + P(t)A - P(t)BR^{-1}B^T P(t) + Q, \quad P(t_f) = P_f. \tag{5.13}$$

The equation is solved backward from t_f. The optimal feedback is

$$u^*(t) = -K(t)x(t), \qquad K(t) = R^{-1}B^T P(t). \tag{5.14}$$

The state then evolves forward under $\dot{x} = (A - BK(t))x$. The Riccati matrix represents future cost, and its terminal condition is a cost specification, not a terminal state value. General actuator constraints invalidate the simple unconstrained gain formula.

5.5 PMP and HJB: methodological comparison

Table 5.1: Source comparison, with the scope of its guarantees made explicit.

Feature	PMP	HJB / dynamic programming
Mathematical form	Coupled Ordinary Differential Equations (ODEs).	Partial Differential Equation (PDE) over time and state.
Usual result	Open-loop optimal candidate $u^*(t)$.	State-feedback policy $u^*(t,x)$.
Optimality	Necessary conditions in the stated setting.	Sufficient verification under suitable assumptions.
Computation	Often lower dimensional; two-point boundary-value problem.	Often high cost: state-space grids suffer the curse of dimensionality in $\mathbb{R}^n$.
Constraints	Input bounds enter Hamiltonian minimization; state bounds need extra conditions.	State/control constraints affect the domain, minimization, and boundary conditions.

The source says PMP handles bounds easily. That is a broad comparison, not a general guarantee of computational ease, particularly for state path constraints. PMP does not inherently forbid deriving feedback; the table describes its usual trajectory-based use.

5.6 Key takeaways

PMP combines forward state evolution with backward adjoint conditions. HJB reasons about optimal remaining cost for every state and yields a feedback policy. LQR is the tractable linear-quadratic case where a matrix Riccati equation replaces the general value-function PDE.

6 Optimal control in discrete time

Source: 2000-opt-ctr-dt-sys.pdf, pp. 1–6; September 27, 2026.

The subtitle is *Formulation, Discrete Maximum Principle, Bellman Equation & DLQR*. The title's maximum-principle terminology and the body heading "Maximum / Minimum Principle" refer to sign conventions; the displayed problem here minimizes cost.

6.1 Finite-horizon problem

Choose the sequence $\{u_0^*, \dots, u_{N-1}^*\}$ over stages $k = 0, \dots, N-1$ to minimize

$$J(u) = \phi(x_N) + \sum_{k=0}^{N-1} L_k(x_k, u_k), \tag{6.1}$$

subject to

$$x_{k+1} = f_k(x_k, u_k), \quad x_0 \text{ given}, \quad u_k \in \Omega_k \subseteq \mathbb{R}^m, \quad x_N \in S_N \subseteq \mathbb{R}^n.$$

The state $x_k \in \mathbb{R}^n$ evolves over a fixed horizon N. The stage cost L_k is incurred at each decision; ϕ is incurred at the terminal state. There are N control actions and $N+1$ states, including both endpoints.

6.2 Discrete adjoint and stationarity conditions

Define the discrete Hamiltonian

$$H_k(x_k, u_k, \lambda_{k+1}) = L_k(x_k, u_k) + \lambda_{k+1}^T f_k(x_k, u_k). \tag{6.2}$$

The multiplier at index $k+1$ is paired with the next state produced at stage k. In the differentiable unconstrained/interior setting, the source conditions are

$$x_{k+1}^* = \nabla_{\lambda_{k+1}} H_k = f_k(x_k^*, u_k^*), \tag{6.3}$$

$$\lambda_k^* = \nabla_{x_k} H_k = \nabla_{x_k} L_k + \left(\frac{\partial f_k}{\partial x_k}\right)^T \lambda_{k+1}^*, \tag{6.4}$$

$$0 = \nabla_{u_k} H_k = \nabla_{u_k} L_k + \left(\frac{\partial f_k}{\partial u_k}\right)^T \lambda_{k+1}^*, \tag{6.5}$$

$$\lambda_N^* = \nabla\phi(x_N^*) \quad \text{for a free terminal state.} \tag{6.6}$$

The state recurrence runs forward; the costate recurrence runs backward. There is no minus sign in equation (6.4), unlike the continuous derivative equation. These are necessary first-order conditions, not by themselves a global minimum test.

Clarification. The zero-gradient condition requires an unconstrained or interior control. At a boundary it must be replaced by the appropriate constrained optimality condition. A terminal constraint changes the terminal multiplier condition. For general nonlinear discrete dynamics, first-order stationarity must not be silently strengthened to an unrestricted global Hamiltonian minimization theorem.

6.3 Bellman recursion and feedback

At stage k, define the remaining optimal cost

$$V_k(x_k) = \min_{\{u_k, \dots, u_{N-1}\}} \left\{ \phi(x_N) + \sum_{i=k}^{N-1} L_i(x_i, u_i) \right\}, \tag{6.7}$$

where the state trajectory follows the dynamics. Bellman's principle decomposes this into current cost plus optimal future cost:

$$V_k(x) = \min_{u \in \Omega_k} \{L_k(x,u) + V_{k+1}(f_k(x,u))\}. \tag{6.8}$$

Start from $V_N(x) = \phi(x)$, then evaluate $k = N-1, N-2, \ldots, 0$. A feedback policy is

$$u_k^*(x) \in \arg\min_{u \in \Omega_k} \{L_k(x,u) + V_{k+1}(f_k(x,u))\}. \tag{6.9}$$

The minimization anticipates the consequence of the action through V_{k+1}; it is not merely greedy minimization of L_k. Although the source subtitle says "value iteration", the displayed finite-horizon algorithm is backward dynamic programming, distinct from iteration toward an infinite-horizon fixed point.

If the original terminal constraint $x_N \in S_N$ is enforced, the boundary can be written as $V_N(x) = \phi(x)$ for $x \in S_N$ and $+\infty$ outside. This encodes infeasibility rather than treating every terminal state as admissible.

6.4 Discrete-time linear quadratic regulator

For $x_{k+1} = Ax_k + Bu_k$, the Discrete-time Linear Quadratic Regulator (DLQR) minimizes

$$J = \tfrac{1}{2} x_N^T P_N x_N + \tfrac{1}{2} \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k). \tag{6.10}$$

Assume symmetric $Q \succeq 0$, $P_N \succeq 0$, and $R \succ 0$, with unconstrained inputs. Use $V_k(x) = \frac{1}{2} x^T P_k x$. Substitution into equation (6.8) gives the one-stage minimization

$$\min_u \left\{ \tfrac{1}{2} x^T Q x + \tfrac{1}{2} u^T R u + \tfrac{1}{2} (Ax + Bu)^T P_{k+1} (Ax + Bu) \right\}.$$

Differentiating with respect to u yields

$$(R + B^T P_{k+1} B) u + B^T P_{k+1} A x = 0.$$

The Hessian $R + B^T P_{k+1} B$ is positive definite, so this is the unique minimum. Thus

$$u_k^* = -K_k x_k, \qquad K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A. \tag{6.11}$$

Symbols: P_{k+1} is the next-stage quadratic cost-to-go matrix; $Q \succeq 0$ penalizes state deviation, $R \succ 0$ penalizes input effort, and P_N is the terminal penalty.

Substituting the minimizing input back into the value function gives the Discrete Difference Riccati Equation (DDRE):

$$\begin{aligned} P_k = {} & Q + A^T P_{k+1} A \\ & - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A. \end{aligned} \tag{6.12}$$

The supplied terminal matrix P_N initializes backward recursion. Once all K_k are known, the state evolves forward with $x_{k+1} = (A - BK_k) x_k$. The input at stage k uses P_{k+1}, not P_k, because the action first changes the next state.

6.5 Continuous and discrete optimal control compared

Table 6.1: The six comparison dimensions in the source's final slide.

Dimension	Continuous time	Discrete time
Dynamics	Differential equation $\dot{x} = f(t, x, u)$.	Difference equation $x_{k+1} = f_k(x_k, u_k)$.
Cost	Integral of L plus terminal cost $\phi(x(t_f))$.	Sum of L_k plus terminal cost $\phi(x_N)$.
Costate	$\dot{\lambda} = -\nabla_x H$.	$\lambda_k = \nabla_{x_k} H_k$.
Dynamic program	HJB partial differential equation.	Bellman functional recurrence.
Riccati system	Differential Riccati equation.	Difference Riccati equation (DDRE).
Implementation	Natural physical-system design formulation.	Natural fit for digital computation and Model Predictive Control (MPC).

The source's physical-versus-digital distinction describes modelling convenience. Continuous-time optimization is also solved numerically, and a discrete controller can act on a physical plant through sampling and a hold device.

6.6 Key takeaways

Finite-horizon dynamic programming works backward from terminal cost, while state simulation works forward from the initial state. In the discrete Hamiltonian, use λ_{k+1}; in DLQR, use P_{k+1}. A terminal state, a terminal cost, and a terminal Riccati matrix are distinct objects.

7 LLM training: supervised fine-tuning and reinforcement learning

Source: 3000-RL4LLM.pdf, pp. 1–15; October 1, 2026.

The lecture, *LLM Training Paradigms: Supervised Fine-Tuning vs. Reinforcement Learning*, is subtitled *Mechanisms, Alignment, Real-World Examples & DeepSeek Case Study*. Enrico Tronci presents it as a Large Language Model Training Overview. Its four themes are supervised instruction pairs, reinforcement learning, comparison of training paradigms, and essential literature.

7.1 From pretraining to supervised instruction following

A large language model (LLM) is first trained to predict successive tokens in a large text corpus. The lecture describes web-scale pretraining on roughly trillions of tokens. This objective builds language-modelling capabilities; it does not by itself specify how an assistant should obey instructions, format a response, or satisfy human preferences. The slide's description of base outputs as "unstructured and unaligned" expresses this missing training objective, not a claim that every base-model output lacks structure.

Supervised Fine-Tuning (SFT) adapts a pretrained model using a dataset $\mathcal{D}_{\text{SFT}}$ of prompt–response pairs (x, y). Here x is the input prompt, including any conversation context, and $y = (y_1, \dots, y_T)$ is the desired response. Unlike the control chapters, x and y now denote token sequences rather than state and measured output.

The instruction pair in the lecture

Prompt x: "Summarize the main mechanism of a Transformer model in two sentences."
Target y (excerpt): "Transformers rely on self-attention mechanisms to process tokens in parallel..."
The slide supplies only this target fragment. The training example illustrates the distinction between an instruction and its demonstrated answer; it does not supply a complete two-sentence reference answer.

Let θ denote the model parameters and $y_{<t} = (y_1, \dots, y_{t-1})$ the target prefix. Autoregressive factorization gives

$$\pi_\theta(y \mid x) = \prod_{t=1}^{|y|} P_\theta(y_t \mid x, y_{<t}), \qquad \log \pi_\theta(y \mid x) = \sum_{t=1}^{|y|} \log P_\theta(y_t \mid x, y_{<t}). \tag{7.1}$$

The target sequence includes the relevant termination convention. The SFT objective in the source is the summed negative log-likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{(x,y)\in\mathcal{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log P_\theta(y_t \mid x, y_{<t}). \tag{7.2}$$

Training uses the demonstrated prefix, a procedure called *teacher forcing*. The model increases the probability of each demonstrated next token conditioned on that prefix. Dividing the entire sum by a fixed total token count rescales this objective without changing its minimizers. Averaging each response separately instead changes the relative weighting of responses of different lengths.

Loss masking. In the output-only convention used by the slide, prompt positions receive no direct next-token loss. They remain conditioning inputs to the response predictions. Consequently, gradients from response losses can still propagate through the computation of prompt representations and through shared parameters. Saying that gradients exist "only on output tokens" should not be interpreted as freezing every computation associated with the prompt.

The source gives 10^4–10^6 high-quality pairs as a typical scale, not a requirement. Human annotation, synthetic instruction generation such as Self-Instruct, and multi-turn conversation logs provide examples. UltraFeedback, also named in the lecture, supplies feedback on candidate responses; selected responses can form SFT targets, while comparisons naturally support preference training. The format of the training objective determines how a dataset is used.

7.2 Historical examples and the role of data quality

The three examples illustrate different balances between model size, dataset size, and curation.

Table 7.1: SFT examples named in the lecture (all from 2023).

Model	Base model	Training data and lesson
Stanford Alpaca	LLaMA-7B	52,000 synthetic instruction demonstrations produced with `text-davinci-003`, using a Self-Instruct-style procedure. Illustrates inexpensive instruction adaptation.
Vicuna	LLaMA	About 70,000 ShareGPT conversations. Multi-turn conversational quality matters; these are conversations, not necessarily 70,000 individual turns.
LIMA (Meta/CMU)	LLaMA-65B	1,000 carefully selected prompt–response examples. Illustrates the potential of a small, high-quality dataset.

LIMA's *Superficial Alignment Hypothesis* proposes that much knowledge and capability is learned during pretraining, while alignment training can largely teach how to present and access it. The reported experiments support this hypothesis in their tested setting. The lecture's word "proved" is too strong: neither a universal theorem nor a guarantee that SFT can only change style follows from those experiments. Dataset quality, coverage, and the starting model all affect the result.

7.3 Why introduce rewards?

Many prompts admit several valid answers. Under equation (7.2), a rephrasing receives no credit merely because its meaning is correct: the loss scores the probabilities of the particular reference tokens. This is a limitation of the supervision signal, not a claim that a model trained by SFT cannot generalize or generate new responses.

A sequence-level reward can evaluate the completed answer: its usefulness, compliance with a safety criterion, or correctness on a mathematical or programming task. These properties need not coincide with matching a single demonstration. A reward is nevertheless a measurement rule or learned proxy; maximizing it establishes only what that proxy actually measures.

Reinforcement learning (RL) allows generated candidates to receive feedback that changes subsequent behaviour. Online sampling can explore strategies absent from the demonstration set. In an explanatory connection to the preceding chapters, the prompt and generated prefix

form a state, choosing the next token is an action, and the policy is the token distribution. Appending a token gives the next state; a terminal reward scores the response. This is a sequential decision problem with a very large state and action space, usually optimized approximately rather than by an exact Bellman table.

7.4 Classic RLHF: demonstrations, preferences, and policy updates

Reinforcement Learning from Human Feedback (RLHF) in the lecture follows three stages: first learn an SFT policy π_{SFT}, then fit a reward model from comparisons, then improve the policy using that reward. InstructGPT is the principal paper example. The slide also names ChatGPT and Claude as assistant families associated with human-feedback alignment; this schematic does not establish an identical PPO implementation for every model or version.

7.4.1 Learning the reward model

A comparison datum is (x, y_w, y_l): for the same prompt x, an annotator prefers the winning response y_w to the losing response y_l. A reward model $r_\psi(x, y) \in \mathbb{R}$, with parameters ψ, assigns a scalar score. With logistic sigmoid $\sigma(z) = 1/(1 + e^{-z})$, the Bradley–Terry preference model is

$$\Pr_\psi(y_w \succ y_l \mid x) = \sigma\big(r_\psi(x, y_w) - r_\psi(x, y_l)\big). \tag{7.3}$$

Its negative log-likelihood is the source's reward-model loss:

$$\mathcal{L}_{\text{RM}}(\psi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\big(r_\psi(x, y_w) - r_\psi(x, y_l)\big)\right]. \tag{7.4}$$

The expectation is an average over labelled comparisons. Only score differences matter: adding the same prompt-dependent constant to both scores leaves the preference probability unchanged. The score is therefore not an absolute measure of truth or utility.

7.4.2 Reward maximization with a reference policy

Let $d(x)$ be the training prompt distribution and let π_{ref} be a fixed reference policy, typically the SFT checkpoint in this classic pipeline. For a positive coefficient β, the regularized objective is

$$J(\theta) = \mathbb{E}_{x\sim d}\left[\mathbb{E}_{y\sim\pi_\theta(\cdot|x)} r_\psi(x, y) - \beta D_{\text{KL}}\big(\pi_\theta(\cdot \mid x) \| \pi_{\text{ref}}(\cdot \mid x)\big)\right]. \tag{7.5}$$

The Kullback–Leibler (KL) divergence compares whole conditional response distributions:

$$D_{\text{KL}}\big(\pi_\theta \| \pi_{\text{ref}}\big)(x) = \sum_y \pi_\theta(y \mid x) \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{7.6}$$

Assume the reference assigns positive probability wherever the policy does and the expectations are finite. Equivalently, J is the expectation over sampled (x, y) of $r_\psi(x, y) - \beta \log[\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x)]$. This distinction corrects the slide's shorthand, which writes KL between scalar probabilities for an individual response.

The reward term favours high-scoring responses; the KL term discourages excessive departure from the reference. Increasing β strengthens this penalty in the objective. It does not impose a hard distance bound or guarantee that training is stable. *Proximal Policy Optimization* (PPO) is an algorithm used to optimize a policy through sampled trajectories and a clipped update surrogate. Equation (7.5) specifies the desired regularized objective, not PPO's full optimization algorithm. The reward model is held fixed during the corresponding policy-optimization stage.

7.5 Direct preference optimization

Direct Preference Optimization (DPO; Rafailov et al., 2023) trains on preference pairs without separately fitting an explicit reward model and without the PPO rollout/value-network loop. Its link to RLHF is a reparameterization of the KL-regularized optimum, not the assertion that DPO performs the same online exploration.

For fixed x, an unrestricted optimal distribution for expected reward minus KL, with $\beta > 0$, has the form

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) e^{r(x,y)/\beta}}{Z(x)}, \qquad r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{7.7}$$

Here $Z(x)$ normalizes the distribution, assumed finite. This explanatory identity follows by imposing normalization in the regularized maximization. In a reward difference for the same prompt, the unknown $\beta \log Z(x)$ cancels. Substituting a trainable policy into the preference model gives the lecture's loss:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]. \tag{7.8}$$

Symbols: π_θ is the trainable policy, π_{ref} the fixed reference policy, (y_w, y_l) the preferred/rejected responses, $\beta > 0$ the preference-scaling coefficient, and σ the logistic sigmoid.

The probabilities are sequence probabilities, computed using sums of token log-probabilities as in equation (7.1). Minimizing the loss encourages a greater relative policy-to-reference likelihood ratio for the preferred response than for the rejected one. It does not require that the absolute probability of the winner increase on every parameter update.

SFT, DPO, and online RL use different feedback

SFT fits a demonstrated response. DPO fits a relative preference between two responses in a dataset. Online RL samples responses and updates a policy using rewards. Standard offline DPO needs no new response sampling during its optimization, although its preference dataset was produced using responses generated earlier. All three can change the same language-model parameters.

The lecture lists Zephyr-7B, Llama-3.1-Instruct, and Mixtral-8x22B-Instruct as adoption examples.

Supplementary clarification: DPO attribution for the named model examples

Zephyr's model card and the Llama 3 report document DPO. The Mixtral-8x22B attribution is retained as a slide example, but the reviewed official model card does not establish that specific training recipe; it should not be confused with Mistral's explicit DPO description of Mixtral-8x7B. Avoid treating an instruct-model name alone as evidence for a particular optimizer. DPO removes major auxiliary training components, but data quality, reference choice, and optimization settings still matter.

7.6 DeepSeek-R1-Zero and group relative policy optimization

The lecture's central case study is DeepSeek-R1-Zero: RL is applied to DeepSeek-V3-Base without a preceding SFT stage. "Without SFT" refers to this post-training initialization; the base model has still undergone extensive pretraining. The policy is improved with *Group Relative Policy Optimization* (GRPO), introduced in DeepSeekMath.

7.6.1 A baseline from a group of answers

For a prompt q (the same role as x above), sample G candidate responses $o_1, \ldots, o_G$ from the old policy π_{old}. Let r_i be the scalar reward of response o_i. The slide defines

$$\bar{r} = \frac{1}{G}\sum_{j=1}^{G} r_j, \qquad \sigma_r = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \bar{r})^2}, \tag{7.9}$$

$$A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon_{\text{norm}}}, \qquad \epsilon_{\text{norm}} > 0. \tag{7.10}$$

Here $\bar{r}$ is the group mean, σ_r the population standard deviation of its rewards, and A_i a group-relative advantage. The small denominator constant avoids division by zero. An answer above the group mean has positive advantage; one below it has negative advantage. A separate learned critic/value network is unnecessary for this baseline.

If all rewards are equal, all A_i are zero: this group supplies no reward-ranking signal. In particular, $G = 1$ is degenerate for these formulas. For illustration, rewards $(0, 1, 1, 0)$ give $\bar{r} = 1/2$, $\sigma_r = 1/2$, and advantages approximately $(-1, 1, 1, -1)$ when ϵ_{norm} is small. An advantage is relative performance within a prompt's sampled group, not an absolute correctness probability.

7.6.2 Clipped policy ratios

For token position t in response o_i, define

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\text{old}}(o_{i,t} \mid q, o_{i,<t})}. \tag{7.11}$$

The old policy is the sampling checkpoint, held fixed during the associated optimization updates; it is distinct from the fixed reference used for KL regularization. The slide's clipped surrogate for a sampled group can then be written legibly as

$$\widehat{J}_{\text{clip}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left\{\rho_{i,t}(\theta)A_i, \operatorname{clip}\left(\rho_{i,t}(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}\right)A_i\right\}. \tag{7.12}$$

Symbols: A_i is the group-relative advantage of response o_i, $\rho_{i,t}$ is the token probability ratio against the old sampling policy, and ϵ_{clip} is the clipping radius.

The clip operation truncates its first argument to the displayed interval; normally $0 < \epsilon_{\text{clip}} < 1$. The group advantage is broadcast to every token of that answer, and $1/|o_i|$ gives the source's per-response length normalization. The two constants ϵ_{norm} and ϵ_{clip} have different purposes even though the slide uses ϵ for both.

For positive A_i, the minimum caps the incentive to increase a ratio beyond $1 + \epsilon_{\text{clip}}$; for negative A_i, it caps the incentive to decrease it below $1 - \epsilon_{\text{clip}}$. This modifies the objective; it does not guarantee that every realized ratio stays within the interval after an optimizer step. Reward scores and computed advantages are treated as fixed when differentiating this surrogate.

Supplementary clarification: What the GRPO slide omits

Equation (7.12) preserves the token-level expression on the slide. It is a sampled clipped surrogate, not a complete specification of GRPO. Training averages over prompts and groups; the cited method also includes a penalty relative to a reference policy. The slide omits both that expectation and KL term. Its formula should therefore not be mistaken for the entire KL-regularized algorithm.

R1-Zero uses rule-based accuracy and formatting rewards. The reported behaviours include longer chain-of-thought (CoT) responses, self-correction, and examples described as "aha moments". These are experimental observations of generated reasoning behaviour, not a proof that all internal reasoning is correct or that such behaviour must emerge in every run. Rewarding a correct final answer or a required format also does not verify every intermediate argument.

7.7 DeepSeek-R1: a hybrid training workflow

R1-Zero exhibited readability problems and language mixing. The R1 workflow in the lecture combines demonstrations and RL to address these issues while developing reasoning and general assistant behaviour. Its four stages are:

1. **Cold-start SFT.** A few thousand curated long-CoT examples initialize readable, structured reasoning responses from the base model.
2. **Reasoning RL.** GRPO with rule-based feedback targets mathematics, coding, and logical tasks with assessable outcomes.
3. **Rejection sampling and further SFT.** Generate candidates, retain suitable reasoning responses, and combine them with general-purpose examples. The slide summarizes this stage as "800k+ high-reward reasoning and general synthetic pairs".

   Supplementary clarification: R1 sample-count breakdown

   The DeepSeek-R1 report describes about 600,000 reasoning and 200,000 non-reasoning samples, approximately 800,000 total. The mixture is therefore not simply a set of rule-verified reasoning answers.

4. **Further RL alignment.** Train for broader helpfulness, safety, and preferences using reward models, while retaining rule-based rewards for reasoning tasks. The final stage is not solely a replacement of verifiable rewards by subjective preferences.

The stages answer different questions: demonstrations provide examples of desired behaviour, rejection sampling selects useful training material, and rewards evaluate candidate outputs. Improved readability and broader alignment are training goals and measured outcomes, not unconditional guarantees about every generated response.

7.8 Comparing SFT, preference fitting, and online RL

The source compares SFT with a column grouping RLHF, DPO, and GRPO. All six dimensions are retained below, with qualifications needed because offline DPO and online RL differ.

Table 7.2: The lecture's comparison, with qualitative claims scoped explicitly.

Dimension	Supervised fine-tuning	Preference optimization / RL
Objective	Token-level demonstration likelihood.	Online RL: sequence reward, usually regularized. DPO: direct preference loss.
Data needed	Prompt–target pairs (x, y).	Preference triples (x, y_w, y_l) and/or reward functions, depending on the method.
Exploration	No online reward-guided exploration in ordinary static SFT; generalization is still possible.	Online RL samples and evaluates candidates. Standard offline DPO does not provide that sampling loop.
Key bottleneck	Demonstration quality, coverage, and the model's ability to learn from them.	Reward hacking, policy drift, and training instability; offline methods also depend on preference-data coverage.
"Reasoning peak"	Slide: "medium", imitating reasoning patterns. No universal upper bound follows.	Slide: "high", discovering reasoning strategies. This is an empirical aspiration, not a universal ranking.
Model examples	Alpaca, Vicuna, LIMA; SFT stages in early LLaMA-1/2-based assistants.	InstructGPT, ChatGPT, Claude, DeepSeek-R1-Zero, DeepSeek-R1; see the attribution limits above.

A model can appear in several training categories because these are stages and objectives, not mutually exclusive model architectures. SFT can learn useful reasoning from strong demonstrations, while an RL-trained model can exploit weaknesses in a reward. Neither a training label nor a high reward by itself constitutes formal verification of the resulting system.

7.9 How the stages fit together

Modern post-training commonly combines SFT with preference optimization or RL. The lecture cites Llama 3, DeepSeek-V3, and GPT-4o as examples of the broader trend. The diagram below makes the stage boundaries explicit without assigning the same disclosed optimizer to all three families.

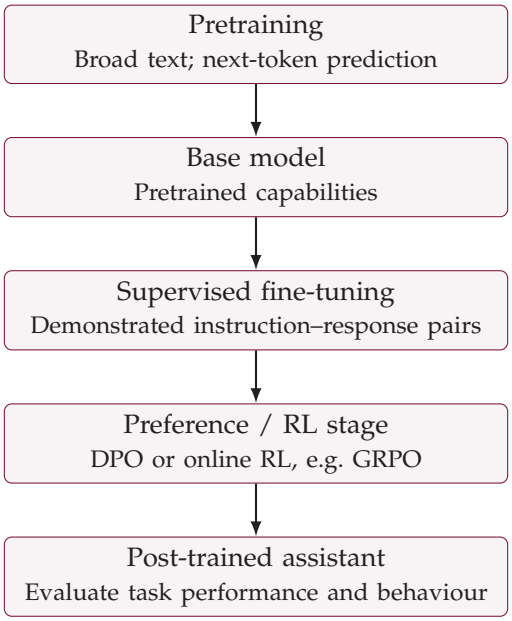


Figure 7.1: A typical combined workflow, shown in chronological order. Pretraining produces the base model; SFT and preference/RL training are subsequent stages. R1-Zero deliberately skips SFT, and other workflows can repeat stages.

The source's arrow marked "Pre-Training / Knowledge Injection" is placed between "Base Model" and "SFT Phase". The redraw clarifies the chronology: a base model is the product of pretraining, and SFT then adapts it. The figure separates processes from resulting checkpoints and gives the preference/RL step its own node.

SFT supports rapid task adaptation, consistent response structure, and specific tool-call syntax. RL can search for effective strategies and improve reward-measured reasoning beyond the supplied demonstrations. Reducing hallucinations and improving adherence to safety guidelines are possible goals, not automatic consequences of using RL. The source's comparison to human demonstrator speed describes an ambition for scalable training, not a proven general speed advantage.

7.10 Key takeaways and literature route

Track four objects separately: the trainable policy, its fixed reference, the old sampling policy, and any reward model. A likelihood target, a human preference, and a rule-based correctness reward are different supervision signals. Distinguish the intended objective from the surrogate used to optimize it, and distinguish experimental improvements from guarantees.

The lecture's reading list moves from FLAN and LIMA (instruction tuning), to InstructGPT and DPO (human preferences), to DeepSeekMath, DeepSeek-V3, and DeepSeek-R1 (GRPO and reasoning). Sutton and Barto supply the foundations of Markov decision processes, value functions, and policy gradients; Jurafsky and Martin cover language-model architectures and training. Full entries for every cited paper and both books appear in Appendix C.4.

8 Exact dynamic programming: deterministic and stochastic

Source: Dimitri P. Bertsekas, Reinforcement Learning and Optimal Control*, Chapter 1, printed pp. 2–17 (PDF pp. 15–30); Sections 1.1–1.2 only, stopping before Section 1.3.*

The textbook introduces exact dynamic programming (DP) as the starting point for the approximate methods that it groups under the reinforcement-learning (RL) methodology. In the terminology of this source, reinforcement learning, approximate dynamic programming, and neuro-dynamic programming refer to closely related approximation-based approaches built on DP. The development begins with finite-horizon problems, first deterministic and then stochastic, because their structure makes the principle of optimality and the backward recursion especially transparent.

Finite-horizon viewpoint

A finite-horizon problem contains a finite sequence of decisions indexed by stages $k = 0, 1, \ldots, N-1$. Dynamic programming solves such a problem through *tail subproblems*: instead of optimizing an entire decision sequence at once, it computes the optimal remaining cost from every relevant intermediate state and works backward from the terminal stage.

8.1 Deterministic dynamic programming

8.1.1 Deterministic finite-horizon problem

A deterministic DP model evolves according to

$$x_{k+1} = f_k(x_k, u_k), \qquad k = 0, 1, \ldots, N-1, \tag{8.1}$$

where x_k is the state, $u_k \in U_k(x_k)$ is the admissible control or decision at stage k, and f_k determines the next state uniquely from (x_k, u_k). The number N is the horizon, i.e. the number of control applications. State and control spaces may depend on the stage and need not be Euclidean or even continuous.

At each stage a cost $g_k(x_k, u_k)$ is incurred, followed by a terminal cost $g_N(x_N)$. For a fixed initial state x_0 and a feasible control sequence $(u_0, \ldots, u_{N-1})$, the total cost is

$$J(x_0; u_0, \ldots, u_{N-1}) = g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k). \tag{8.2}$$

Because the system is deterministic, x_0 together with the control sequence fixes the complete state trajectory. The optimal cost is therefore

$$J^*(x_0) = \min_{\substack{u_k \in U_k(x_k) \\ k=0,\ldots,N-1}} J(x_0; u_0, \ldots, u_{N-1}). \tag{8.3}$$

The source's minimization convention

The textbook writes "min" even when attainment has not been established, using it as notation for the infimum over the feasible controls. We retain that convention for values. Any construction using arg min additionally assumes that a minimizing control exists; computing an infimum alone does not establish that assumption.

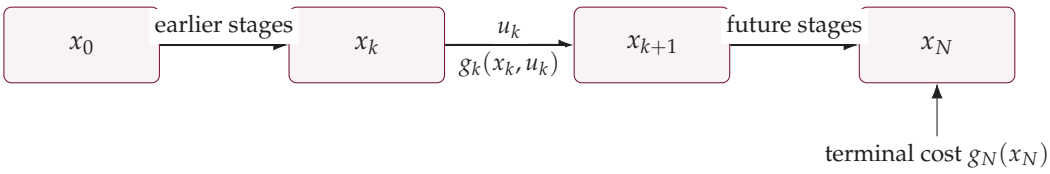


Figure 8.1: Deterministic N-stage optimal-control structure from the source's Fig. 1.1.1: a control selects the next state and incurs an additive stage cost; the process ends with a terminal cost.

8.1.2 Finite-state problems as shortest paths

When states and controls take finitely many values, the deterministic problem can be represented as a layered acyclic graph. Nodes are states x_k and an arc corresponds to a state–control pair (x_k, u_k) leading to $x_{k+1} = f_k(x_k, u_k)$. The arc length is the stage cost $g_k(x_k, u_k)$. An artificial terminal node can be attached to every state x_N by an arc whose length is $g_N(x_N)$. A feasible control sequence is then a path, and minimizing total cost is equivalent to finding a shortest path from the initial node to the terminal node.

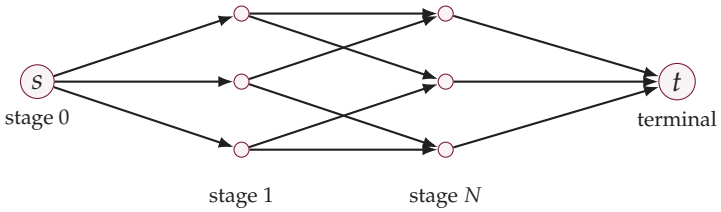


Figure 8.2: Layered transition-graph interpretation of a deterministic finite-state problem. Arc lengths represent stage costs, and terminal arcs can encode g_N.

8.1.3 Scheduling example

The source illustrates the graph formulation with four operations A, B, C, D. Operation B may be performed only after A, while D may be performed only after C. The state records the ordered prefix of operations already completed, and a control selects the next admissible operation. Equivalently, a compact state consists of the completed-operation set together with the last operation. The set alone is insufficient: prefixes AC and CA leave the same operations unfinished but have different next setup costs. Setup costs C_{mn} are incurred when moving from operation m to operation n, together with a startup cost when the first operation is selected. For example, the sequence $ACDB$ has cost

$$S_A + C_{AC} + C_{CD} + C_{DB}.$$

The graph is deterministic because each admissible decision identifies the next state with certainty. In the source graph, three decisions select the first three operations; the remaining operation is forced and its setup cost is the terminal cost. A four-decision formulation with zero terminal cost is equivalent.

The same example is then solved explicitly by backward DP. The cost-to-go values reported by the source are summarized below.

Table 8.1: Backward solution of the four-operation scheduling example.

Tail state	Comparison used by DP	Optimal cost-to-go
AB	Only C can be selected next; completing with D gives $3+6$.	9
AC	Complete as BD with cost 5, or as DB with cost 9.	5
CA	Complete as BD with cost 3, or as DB with cost 7.	3
CD	Only A can be selected next; the remaining completion cost is 5.	5
A	Choose B: $2+9=11$; choose C: $3+5=8$.	8
C	Choose A: $4+3=7$; choose D: $6+5=11$.	7
Initial state	Start with A: $5+8=13$; start with C: $3+7=10$.	10

Following the minimizing decisions forward yields the optimal schedule $CABD$ with total cost 10. The example shows the characteristic DP pattern: solve short tail problems first, reuse their optimal costs, and then reconstruct an optimal sequence by following minimizing decisions forward.

8.1.4 Continuous-space linear–quadratic example

The second deterministic example passes a material through N ovens. Let x_0 be the initial temperature, x_k the temperature after oven k, and u_k the heat energy applied at stage k. The scalar dynamics are

$$x_{k+1} = (1-a)x_k + au_k, \qquad 0 < a < 1, \tag{8.4}$$

and the objective is to approach a target temperature T while limiting energy expenditure:

$$r(x_N - T)^2 + \sum_{k=0}^{N-1} u_k^2, \qquad r > 0. \tag{8.5}$$

The example is deliberately unconstrained for analytical tractability, while the source notes that practical bounds on states or controls may also be imposed. Such constrained deterministic continuous-space problems may be attacked not only by DP but also by quadratic or nonlinear-programming methods.

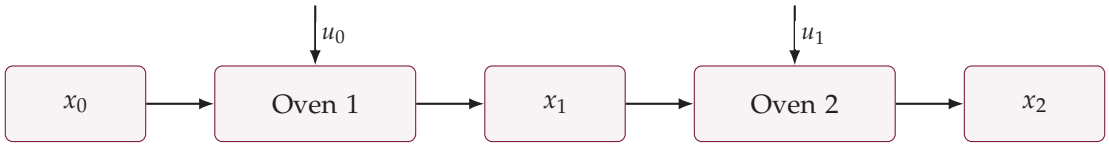


Figure 8.3: The source's $N = 2$ oven interpretation of the scalar linear–quadratic example.

8.2 The dynamic programming algorithm

8.2.1 Principle of optimality

The central structural statement is Bellman's principle of optimality: if

$$\{u_0^*, u_1^*, \ldots, u_{N-1}^*\}$$

is optimal from x_0 and generates the optimal trajectory $\{x_1^*, \ldots, x_N^*\}$, then for every intermediate stage k the truncated sequence

$$\{u_k^*, u_{k+1}^*, \ldots, u_{N-1}^*\}$$

is optimal for the tail problem that starts from x_k^* at stage k. Otherwise, replacing the tail by a better feasible tail would improve the original sequence and contradict optimality. The source gives the intuitive route analogy that if the fastest route from Los Angeles to Boston passes through Chicago, the Chicago-to-Boston suffix must itself be fastest for that subtrip.

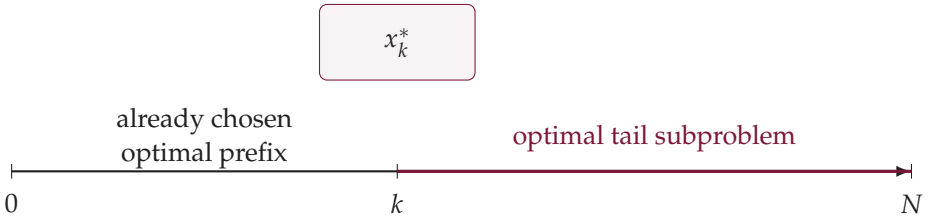


Figure 8.4: Principle of optimality: the tail of an optimal control sequence is optimal for the tail subproblem reached along the optimal trajectory.

This principle suggests a backward construction: solve the one-stage tail problems first, then the two-stage tails, and continue until the full horizon has been covered.

8.2.2 Backward recursion and cost-to-go

Define the terminal cost-to-go by

$$J_N^*(x_N) = g_N(x_N). \tag{8.6}$$

For $k = N-1, N-2, \ldots, 0$, the deterministic DP recursion is

$$J_k^*(x_k) = \min_{u_k \in U_k(x_k)} \left[g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)) \right]. \tag{8.7}$$

Here $J_k^*(x_k)$ is the optimal cost accumulated from state x_k at stage k to the terminal stage:

$$J_k^*(x_k) = \min_{\substack{u_m \in U_m(x_m) \\ m=k,\ldots,N-1}} J(x_k; u_k, \ldots, u_{N-1}), \tag{8.8}$$

$$J(x_k; u_k, \ldots, u_{N-1}) = g_N(x_N) + \sum_{m=k}^{N-1} g_m(x_m, u_m). \tag{8.9}$$

The Bellman equation follows by separating the first stage from the remaining tail:

$$\begin{aligned} J_k^*(x_k) &= \min_{u_k \in U_k(x_k)} \left[g_k(x_k, u_k) + \min_{u_{k+1}, \ldots, u_{N-1}} \left(g_N(x_N) + \sum_{m=k+1}^{N-1} g_m(x_m, u_m) \right) \right] \\ &= \min_{u_k \in U_k(x_k)} \left[g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)) \right]. \end{aligned}$$

Thus the full multistage problem is reduced to a sequence of one-stage minimizations linked through the cost-to-go.

Extended-real caveat from the source

The textbook notes that, through minimization, some cost-to-go values may be $-\infty$. The induction argument establishing the DP recursion can still be interpreted in that extended-real setting.

8.2.3 Constructing an optimal control sequence

Once all functions $J_0^*, \dots, J_N^*$ have been computed backward, an optimal trajectory is constructed forward, provided the required minima are attained. Starting at the given x_0, choose

$$u_k^* \in \arg\min_{u_k \in U_k(x_k^*)} \left[g_k(x_k^*, u_k) + J_{k+1}^*(f_k(x_k^*, u_k)) \right], \tag{8.10}$$

then propagate

$$x_{k+1}^* = f_k(x_k^*, u_k^*), \qquad k = 0, 1, \dots, N-1. \tag{8.11}$$

The key computational asymmetry is therefore: *cost-to-go functions are computed backward, while the selected state/control trajectory is generated forward*.

8.3 Approximation in value space and Q-factors

Exact DP requires the optimal cost-to-go $J_k^*(x_k)$ for all relevant states and stages, which may be prohibitively expensive when the state space is large. The source therefore introduces the approximation idea that motivates later RL methods: replace J_k^* by a tractable approximation $\widetilde{J}_k$ and use the same one-step lookahead structure.

The approximate forward decision rule is

$$\widetilde{u}_k \in \arg\min_{u_k \in U_k(\widetilde{x}_k)} \left[g_k(\widetilde{x}_k, u_k) + \widetilde{J}_{k+1}(f_k(\widetilde{x}_k, u_k)) \right], \tag{8.12}$$

with

$$\widetilde{x}_{k+1} = f_k(\widetilde{x}_k, \widetilde{u}_k). \tag{8.13}$$

The resulting solution is generally suboptimal because $\widetilde{J}_k$ need not equal the exact cost-to-go.

Define the approximate Q-factor

$$\widetilde{Q}_k(x_k, u_k) = g_k(x_k, u_k) + \widetilde{J}_{k+1}(f_k(x_k, u_k)). \tag{8.14}$$

Then the approximate action selection becomes simply

$$\widetilde{u}_k \in \arg\min_{u_k \in U_k(\widetilde{x}_k)} \widetilde{Q}_k(\widetilde{x}_k, u_k).$$

For exact DP, the optimal Q-factor is

$$Q_k^*(x_k, u_k) = g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)), \tag{8.15}$$

and the optimal cost-to-go is recovered by

$$J_k^*(x_k) = \min_{u_k \in U_k(x_k)} Q_k^*(x_k, u_k). \tag{8.16}$$

Initialize the last-stage Q-factor directly from the terminal cost:

$$Q_{N-1}^*(x, u) = g_{N-1}(x, u) + g_N(f_{N-1}(x, u)). \tag{8.17}$$

Eliminating J_{k+1}^* then yields a Q-factor-only form of the deterministic DP recursion for $k = N-2, \dots, 0$:

$$Q_k^*(x_k, u_k) = g_k(x_k, u_k) + \min_{u_{k+1} \in U_{k+1}(f_k(x_k, u_k))} Q_{k+1}^*(f_k(x_k, u_k), u_{k+1}). \tag{8.18}$$

Terminology used by the source

The book uses *Q-factor*; common alternatives are *action value*, *state–action value*, and *Q-value*. It credits Watkins's thesis with introducing the term "Q-learning" and associated algorithmic ideas. The present excerpt introduces the exact and approximate Q-factor structure; later chapters develop the simulation-based learning algorithms.

8.4 Stochastic dynamic programming

8.4.1 Random dynamics and feedback policies

The stochastic finite-horizon model introduces a disturbance w_k:

$$x_{k+1} = f_k(x_k, u_k, w_k), \qquad k = 0, 1, \ldots, N-1. \tag{8.19}$$

The disturbance has a conditional distribution $P_k(\cdot \mid x_k, u_k)$ that may depend on the current state and control but, in the source's model, not on the earlier history once (x_k, u_k) are specified. All expectations below use this conditional distribution at the displayed state and action. The stage cost may also be random through

$$g_k(x_k, u_k, w_k).$$

The optimization variable is now a *policy*, not merely a fixed open-loop control sequence. A policy is

$$\pi = \{\mu_0, \mu_1, \ldots, \mu_{N-1}\}, \qquad u_k = \mu_k(x_k), \tag{8.20}$$

with $\mu_k(x_k) \in U_k(x_k)$ for every admissible state. This feedback structure allows the controller to react to the state actually reached after random disturbances, which is the fundamental distinction emphasized by the source.

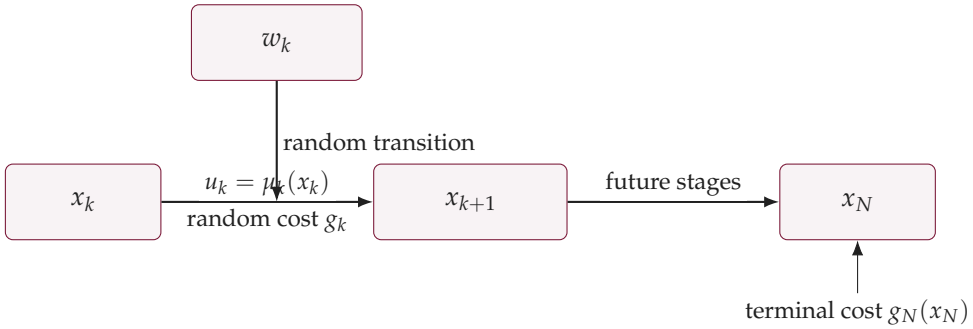


Figure 8.5: Stochastic N-stage structure from the source's Fig. 1.2.1: the disturbance affects both the transition and potentially the stage cost, so the realized future state is random.

Given initial state x_0 and policy π, the induced state and disturbance sequences are random. The expected total cost is

$$J_\pi(x_0) = \mathbb{E}\left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), w_k) \right]. \tag{8.21}$$

If Π denotes the admissible policies, then

$$J^*(x_0) = \min_{\pi \in \Pi} J_\pi(x_0). \tag{8.22}$$

The source notes that these expected values may require simulation, including Monte Carlo methods, in contrast with the completely deterministic trajectory evaluation above.

8.4.2 Stochastic finite-horizon DP recursion

The stochastic algorithm retains the same tail-subproblem structure as deterministic DP, but introduces an expectation inside each one-stage minimization. Initialize

$$J_N^*(x_N) = g_N(x_N), \tag{8.23}$$

and compute backward for $k = N-1, N-2, \ldots, 0$:

$$J_k^*(x_k) = \min_{u_k \in U_k(x_k)} \mathbb{E}\left[g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))\right]. \tag{8.24}$$

If a minimizing selector $u_k^* = \mu_k^*(x_k)$ is chosen for every x_k and k, then

$$\pi^* = \{\mu_0^*, \ldots, \mu_{N-1}^*\}$$

is an optimal policy. As in the deterministic case, $J_0^*(x_0)$ equals the optimal cost from the initial state.

The source highlights four features shared by deterministic and stochastic finite-horizon DP:

1. multistage optimization is decomposed into one-stage minimizations over tail subproblems;
2. cost-to-go functions are generated backward for every relevant state;
3. an optimal decision rule is extracted from the minimizers in the DP equations;
4. replacing exact cost-to-go functions by approximations yields a natural approximation-in-value-space scheme.

Well-defined expectations

The source notes a technical requirement that the expected values in the stochastic DP equations be well defined and finite. This issue disappears in the elementary finite-disturbance case when all expectations reduce to finite sums of real terms.

8.4.3 Stochastic Q-factors and simulation

The stochastic optimal Q-factor is the expression minimized in equation (8.24):

$$Q_k^*(x_k, u_k) = \mathbb{E}\left[g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k))\right]. \tag{8.25}$$

As before,

$$J_k^*(x_k) = \min_{u_k \in U_k(x_k)} Q_k^*(x_k, u_k). \tag{8.26}$$

Initialize the last-stage Q-factor by

$$Q_{N-1}^*(x, u) = \mathbb{E}[g_{N-1}(x, u, w_{N-1}) + g_N(f_{N-1}(x, u, w_{N-1}))]. \tag{8.27}$$

Substitution then gives the Q-factor recursion for $k = N-2, \ldots, 0$:

$$\begin{aligned} Q_k^*(x_k, u_k) = \mathbb{E}\Bigg[& g_k(x_k, u_k, w_k) \\ & + \min_{u_{k+1} \in U_{k+1}(f_k(x_k, u_k, w_k))} Q_{k+1}^*(f_k(x_k, u_k, w_k), u_{k+1}) \Bigg]. \end{aligned} \tag{8.28}$$

The source emphasizes that the expectation in this Q-factor form can be approximated naturally by sampling and simulation, a point that becomes central later in simulation-based Q-factor

algorithms. The next control is chosen after observing the next state, which is why its minimization is inside the expectation. Moving the minimization outside would require choosing that control before the disturbance is observed and would generally describe a different problem. For both Q-factor recursions, the last-stage initialization avoids introducing an undefined terminal action u_N or Q-factor Q_N^*.

8.5 Deterministic and stochastic DP compared

Table 8.2: Finite-horizon deterministic and stochastic DP in the assigned Bertsekas excerpt.

Feature	Deterministic	Stochastic
Dynamics	$x_{k+1} = f_k(x_k, u_k)$.	$x_{k+1} = f_k(x_k, u_k, w_k)$.
Optimization object	Feasible control sequence.	Feedback policy $\pi = \{\mu_k\}$.
Stage cost	$g_k(x_k, u_k)$.	$g_k(x_k, u_k, w_k)$ may be random.
Bellman step	Minimize current cost plus exact next cost-to-go.	Minimize the expected current cost plus next cost-to-go.
Execution	State trajectory is fixed once x_0 and controls are fixed.	Realized state is random; feedback adapts controls to the observed state.
Approximation link	Replace J_k^* by $\widetilde{J}_k$ or approximate Q-factors.	Same idea, with expectations that may be estimated by simulation.

8.6 Key takeaways from the assigned excerpt

Exact finite-horizon DP is organized around three objects: the state, the additive cost, and the optimal cost-to-go. Bellman's principle of optimality makes backward recursion possible. In deterministic problems the optimization can be viewed as shortest-path search when the spaces are finite. In stochastic problems the decision variable is a feedback policy and the Bellman step includes an expectation. Approximation in value space replaces J_k^* by $\widetilde{J}_k$; Q-factors package the one-step cost and future cost into a state–action quantity. These constructions are the bridge from exact optimal control to the approximation and simulation methods that the source later calls reinforcement learning.

9 Numerical integration: a simple summary

Source: François E. Cellier and Ernesto Kofman, Continuous System Simulation*, Chapter 2, Sections 2.1–2.4, printed pp. 25–42; stopping before Section 2.5.*

This chapter summarizes the assigned reading in simple terms. The subsection titles retain the book's section numbers so that they are easy to match to the source.

9.1 Book 2.1: Introduction

A continuous-time model tells us how fast the state changes:

$$\dot{x}(t) = f(x(t), u(t), t), \qquad x(t_0) = x_0.$$

Here x is the state, u is the input, and x_0 is the starting value. For example, if x is a temperature, $\dot{x}$ tells us how quickly it rises or falls.

The computer estimates the state at separate times $t_0, t_0 + h, t_0 + 2h, \ldots$, where $h > 0$ is the **step size**. A numerical integration method is a rule for estimating the next state from the model and the information already available.

For a sufficiently smooth trajectory, a Taylor expansion explains the idea:

$$x(t + h) = x(t) + h\dot{x}(t) + \frac{h^2}{2}\ddot{x}(t) + \cdots .$$

The simplest estimate keeps just the current value and the first change: $x(t + h) \approx x(t) + h\dot{x}(t)$. More advanced methods account for more of the trajectory's curvature. A finite Taylor approximation requires sufficient smoothness; an infinite Taylor series need not represent every smooth function.

9.2 Book 2.2: The approximation accuracy

Accuracy means how close the computed trajectory is to the exact solution of the model. The book distinguishes three numerical errors:

- **Truncation error:** we simplify the update, for example by dropping higher Taylor terms.
- **Roundoff error:** the computer stores numbers with limited precision, so tiny changes can lose digits when added to a large state value.
- **Accumulation error:** each step starts from a previously computed value, so earlier errors affect later steps. The dynamics can damp or amplify them.

A method of order p typically has a one-step error of size $O(h^{p+1})$ when that step starts from the exact state. Over a fixed time interval, its total error is $O(h^p)$ under the usual smoothness and stability assumptions, for sufficiently small h. The notation $O(h^p)$ describes how the error scales as h becomes small; it does not give the exact error.

For example, halving h roughly halves the total error of a first-order method, while it roughly divides the total error of a second-order method by four. Higher-order methods cost more per step, but may need fewer steps. Smaller steps generally reduce truncation error, though very small steps can make roundoff more significant. The book also discusses storing the state at higher precision than intermediate calculations as a compromise between precision and cost.

Two separate checks

Model validation: does the model describe the real system well enough? Model errors include incorrect parameters and missing dynamics.
Simulation verification: does the numerical computation solve that model accurately enough? Even a very accurate computation cannot repair a poor model.

9.3 Book 2.3: Euler integration

Write $x_k \approx x(t_k)$ and $t_{k+1} = t_k + h$. For simplicity, the formulas below suppress the input and write the model as $\dot{x} = f(x, t)$.

Forward Euler (FE) uses the slope at the start of the step:

$$\boxed{x_{k+1} = x_k + hf(x_k, t_k).}$$

Think of walking along the current tangent for a short time. Everything on the right is known, so the update is **explicit**: evaluate the slope, multiply by h, and add the result to x_k.

Backward Euler (BE) uses the slope at the end of the step:

$$\boxed{x_{k+1} = x_k + hf(x_{k+1}, t_{k+1}).}$$

The unknown next state appears on both sides, so the update is **implicit**: solve an equation to find x_{k+1}. For nonlinear models, this usually needs an iterative solver. "Backward" refers to the slope evaluation; the simulation still advances forward in time. Both Euler methods are first order.

For $\dot{x} = -2x$, starting from $x_0 = 1$ with $h = 0.1$:

$$\begin{aligned} &\text{FE:} \quad x_1 = 1 + 0.1(-2) = 0.8, \\ &\text{BE:} \quad x_1 = 1 - 0.2x_1 \quad \Longrightarrow \quad x_1 = \frac{1}{1.2} \approx 0.8333. \end{aligned}$$

The exact value is $e^{-0.2} \approx 0.8187$. Both are approximations; changing the slope evaluation changes the result.

9.4 Book 2.4: The domain of numerical stability

A stable continuous model can have an unstable numerical simulation. To see why, use the simple test equation $\dot{x} = \lambda x$. Its exact solution is $x(t) = e^{\lambda t} x_0$, taking $t_0 = 0$. If $\operatorname{Re} \lambda < 0$, the exact state decays to zero: this is *asymptotic* stability.

A numerical method gives an update $x_{k+1} = R(z)x_k$, where $z = h\lambda$ and R is the step's multiplication factor. The computed state decays only when

$$\boxed{|R(z)| < 1.}$$

The **stability domain** is the set of z values satisfying this inequality.

Forward Euler: $R(z) = 1 + z$, so its domain is

$$|1 + z| < 1.$$

In the complex plane, this is the inside of a circle of radius 1 centered at -1. For a real negative λ, it requires

$$0 < h < \frac{2}{|\lambda|}.$$

For example, $\dot{x} = -3x$ decays in reality, but with $h = 1$, FE gives $x_{k+1} = -2x_k$: the simulation alternates sign and grows. Reducing the step below $2/3$ makes it decay. At $h = 2/3$ it alternates without decaying. Even a stable step may be too large for accurate results.

Backward Euler: $R(z) = 1/(1 - z)$, so its domain is

$$|1 - z| > 1.$$

This is the outside of a circle of radius 1 centered at 1. It contains the whole left half-plane, so BE preserves decay for every $h > 0$ when $\operatorname{Re}\lambda < 0$. This helps with **stiff systems**, which combine fast-decaying and slow dynamics: FE may require extremely small steps just to remain stable. BE still needs an appropriate step for accuracy.

BE can also hide real instability if the step is too large. For $\dot{x} = 3x$ and $h = 1$, it gives $x_{k+1} = -x_k/2$, which decays even though the exact solution grows. For undamped oscillations, with nonzero purely imaginary λ, FE produces artificial growth and BE produces artificial damping.

For a linear system $\dot{x} = Ax$, the update matrices are

$$F_{\text{FE}} = I + hA, \qquad F_{\text{BE}} = (I - hA)^{-1},$$

where I is the identity matrix and the BE inverse must exist. All eigenvalues of F must have magnitude less than 1 for numerical asymptotic stability. Equivalently, check every $h\lambda_i$ against the method's domain. The book illustrates how to trace its boundary numerically: vary an eigenvalue's angle, adjust h, and locate where the largest eigenvalue magnitude of F reaches 1.

What to remember

A smaller step usually improves accuracy. FE is easy to compute but has a restricted stability domain. BE requires solving an equation and handles fast decay better. **Stability does not guarantee accuracy.** Compare results using smaller steps and different methods to check whether the simulation is dependable; agreement supports verification but is not a proof.

10 Quick review

10.1 Core distinctions

Question	What to check
What is the state?	Variables sufficient to propagate the model, not necessarily just the output.
What does the input affect?	Next frame or next sample; do not shift input/measurement pairs.
Is a model exact?	Separate assumed plant structure, learned parameters, exact sampling, and approximations.
Is the origin asymptotically stable?	Continuous LTI: $\operatorname{Re}\lambda_i < 0$. Discrete LTI: $\lvert\lambda_i\rvert < 1$.
Is it optimal?	PMP supplies necessary conditions; Bellman/HJB characterize optimal future cost under their assumptions.
What runs backward?	Costates, value functions, and finite-horizon Riccati equations.
What supervises the LLM?	Demonstrated targets (SFT), pairwise preferences (DPO/RLHF), or scores on generated outputs (online RL).
Which policy is fixed?	Distinguish the reference policy from the old sampling policy; both differ from the trainable policy.
Deterministic or stochastic DP?	Deterministic DP optimizes a control sequence; stochastic DP optimizes a state-feedback policy and places an expectation inside the Bellman step.
Cost-to-go or Q-factor?	$J_k^*(x_k)$ values a state from stage k onward; $Q_k^*(x_k, u_k)$ also conditions on the current action.

10.2 Exam revision prompts

1. Derive the sliding-window gain estimate and state exactly when its denominator vanishes.
2. Starting from the two compressor state equations, compute the determinant and explain why one pole is zero.
3. Explain why $K = (1-p)/\widehat{\alpha}$ places a nominal pole but does not establish an unconditional adaptive guarantee.
4. Derive both Euler and exact ZOH motor updates. Identify the missing damping term in the source's approximate displacement.
5. Explain the transpose in PageRank and why a stationary eigenvalue 1 is compatible with convergence of probability distributions.
6. State the continuous and discrete stability criteria, and derive the Lyapunov energy decrement.
7. Derive the adjoint equations using the correct sign or index for each time domain.
8. Starting from Bellman's equation, derive the DLQR gain and explain the use of P_{k+1}.
9. State the SFT, reward-model, and DPO losses, with datasets and trainable parameters.
10. Explain KL between distributions and the sampled log-ratio form of the RLHF objective.
11. Compute group-normalized advantages, including the equal-reward case.
12. Contrast clipping with hard constraints, and R1-Zero with the four-stage R1 workflow.
13. State the deterministic finite-horizon Bellman recursion and explain why it is evaluated backward.
14. Starting from Bellman's equation, derive the deterministic Q-factor recursion and recover J_k^* from Q_k^*.
15. Explain why stochastic optimal control optimizes feedback policies rather than fixed control sequences.
16. Write the stochastic Bellman recursion and identify exactly where the expectation appears.
17. Explain why the stochastic Q-factor formulation is especially compatible with sampling and simula-

tion.

10.3 Compact terminology

Plant: controlled environment or process. **Reference:** desired output. **Feedback:** use of measured behaviour in selecting inputs. **Online identification:** model estimation during operation. **Equilibrium:** constant state under a compatible constant input. **Schur stable:** all eigenvalues strictly inside the unit disk. **Costate:** adjoint multiplier for dynamic constraints. **Value function:** optimal remaining cost. **Riccati equation:** matrix equation governing a quadratic value function in LQR. **ZOH:** input held constant between samples. **SFT:** supervised fine-tuning on demonstrated responses. **RLHF:** reinforcement learning from human feedback. **DPO:** direct preference optimization. **GRPO:** group relative policy optimization. **CoT:** chain of thought. **KL divergence:** a directional measure of distribution mismatch. **Dynamic programming (DP):** backward solution of multistage decision problems through tail subproblems. **Cost-to-go J_k^*:** optimal remaining cost from state x_k at stage k. **Q-factor Q_k^*:** optimal cost associated with a state–action pair before minimizing over the action. **Policy:** state-dependent control law; in the stochastic finite-horizon setting, $\pi = \{\mu_0, \ldots, \mu_{N-1}\}$.

A Corrections and source ambiguities

This register records substantive changes and assumptions. Spelling and malformed slide totals have been normalized without changing academic content. Additional derivations are identified in the main text.

Source	Treatment in these notes
Introduction p. 3	The registration instruction names “Software Verification”. Preserved as an unresolved course-name mismatch. Timetable and assessment are reported as source information.
Compressor p. 3	The $[0, 1]$ SSIM scale is retained as the case-study convention, not a universal range for general SSIM. The orientation and feasible range of the compressor setting are unspecified.
Compressor pp. 8–10	The displayed objective is a sum, not a mean, of squared errors. Both have the same minimizer. A nonzero input-energy denominator is required.
Compressor pp. 11–12	The stated control law is integral-only despite the PI label. The sum identity assumes a fixed gain.
Compressor pp. 20–24	The characteristic polynomial has zero constant term. Correct poles: 0 and $1 - \alpha K$. The original erroneous two-pole argument is explicitly reconstructed and diagnosed in section 2.8; its interval reparameterization is also invalid. The final gain formula remains valid by the corrected derivation.
Compressor pp. 23–27	The estimate is distinguished from the true gain. Fixed-model pole placement is not a general adaptive-stability proof. Warm-up settings and accumulator treatment are unspecified; the learner also needs past control settings.
Continuous systems p. 5	Lyapunov stability means remaining close, not merely bounded. Asymptotic stability includes stability plus attraction. Controllability is qualified by admissible-input constraints.
Continuous systems p. 6	The car model’s five components are not fully specified. No fifth variable is invented. Non-holonomic is a motion constraint.
Discrete systems pp. 5–6	Exact ZOH is separated from leading-order input approximations. A consistent second-order angle expansion includes damping. Constant held voltage does not generally mean constant acceleration.
Discrete systems pp. 7–8	Stable age-profile convergence requires suitable matrix assumptions. Growth factor and normalized age distribution are distinguished.
Discrete systems pp. 9–10	Dangling pages need stochastic rows before damping. Stationary PageRank is not stability of the origin in the full vector space.
Discrete systems p. 11	$\mathcal{R}_0$ describes initial growth; later growth depends on S_k. Parameter scaling and nonnegative-step conditions are clarified.
Discrete systems pp. 12–14	Exact-sampling stability differs from Euler stability. Lyapunov strict inequalities exclude zero. The open-loop motor has an angle integrator. Taxonomy interventions are not present in the displayed models.
Continuous optimal control pp. 2–4, 6	Normality, smoothness, admissibility, and endpoint assumptions are made explicit. Necessary PMP conditions are distinguished from HJB verification. State constraints are not universally easy to handle.

Source	Treatment in these notes
Both LQR slides, p. 5	Standard definiteness assumptions and unconstrained inputs are stated. Riccati signs, terminal conditions, and discrete $k+1$ indexing are preserved.
Discrete optimal control pp. 2–4	Zero-gradient stationarity requires an interior/unconstrained input. A terminal set changes the boundary condition. Finite-horizon backward recursion is distinguished from infinite-horizon value iteration.
LLM p. 4	Output masking removes prompt-position losses, not all gradient paths through prompt representations. Dataset scales are illustrative.
LLM p. 5	Vicuna's count denotes conversations. LIMA supports a hypothesis experimentally; it does not prove that SFT only teaches style.
LLM pp. 7–8	KL acts on response distributions. The RLHF objective is distinguished from PPO's algorithm; offline DPO is separated from online exploration. Mixtral-8x22B's DPO attribution is not established by the reviewed official card.
LLM p. 9	Distinct normalization and clipping constants are used. The source expression is identified as a sampled clipped surrogate that omits the training expectation and KL penalty. Equal group rewards give zero advantages.
LLM p. 10	R1's later SFT mixture is approximately 600k reasoning plus 200k general samples. Final RL combines rule-based and learned reward signals.
LLM pp. 11–12	"Zero/high" exploration and "medium/high" reasoning are qualified; no universal capability bound follows. The diagram places pretraining before the base checkpoint. Training goals are not guarantees.
LLM p. 15	Jurafsky–Martin's 2024 citation is treated as a third-edition draft; chapter numbers vary with the draft.
Bertsekas scheduling example	The state retains the last operation as well as the completed set; AC and CA must remain distinct because their future setup costs differ. The source graph puts the forced fourth operation's setup cost in the terminal cost.
Bertsekas DP and Q-factors	Value infima do not guarantee attained controls. Both Q-factor recursions start from an explicit last-stage boundary and recurse only through $k = N-2, \ldots, 0$. Stochastic expectations are conditional on the current state/action; next-stage minimization follows observation of the next state.

B Source coverage map

Every lecture-deck page and every assigned textbook page used in these notes is accounted for below. Title and closing pages are tracked separately from academic content. Repeated derivation steps are consolidated without losing their equations or conclusions. The compressor architectures are redrawn in figures 2.1 to 2.3; the LLM training pipeline is redrawn in figure 7.1. Equations and comparison tables are typeset directly. Current optimal-control filenames replace their identical earlier aliases, without duplicating source-page counts.

File	Page	Content destination
1000-intro	1	Title page; course identity in Section 1.1.
	2	Section 1.1: timetable, room, address.
	3	Section 1.1: platform, recordings, registration ambiguity, contact.
	4	Section 1.1: assessment weights and individual pass thresholds.
	5	Section 1.1: full course-material list.
	6	Section 1.2: all four objectives.
	7	Section 1.2: closing acknowledgement; no additional content.
1500-video-compressor	1	Section 2 opening and 2.1: title, identity, research URL.
	2	Section 2.1: full case-study reference.
	3	Section 2.1: bandwidth–quality trade-off, SSIM, image dependence.
	4	Section 2.2 and Figure 2.1: detailed plant and online learning.
	5	Section 2.2 and Figure 2.2: abstract plant and planning.
	6	Section 2.3 and Figure 2.2: signal timing and plant equation.
	7	Section 2.3: reference 0.9 and parameter-learning rationale.
	8	Section 2.4: sliding-window measurements, predictions, objective.
	9	Section 2.4: all derivative steps and stationarity.
	10	Section 2.4: estimator and known-quantity interpretation.
	11	Section 2.5: error, integral law, positive gain, PI terminology.
	12	Section 2.5: accumulator, initialization, sum identity.
	13	Sections 2.5–2.6: closed loop and input recurrence.
	14	Section 2.6: error-based and reference-based state equations.
	15	Section 2.6: expanded two-state equations.
	16	Sections 2.6–2.7: tracking and asymptotic-stability design goals.
	17	Section 2.6: equilibrium substitutions.
	18	Section 2.6: equilibrium solution and regulator interpretation.
	19	Sections 2.6–2.7: system matrix and eigenvalue criterion.
	20	Sections 2.7–2.8: determinant, corrected polynomial, source error.
	21	Section 2.8: original coefficient comparison and missing factor.
	22	Section 2.8: original pole relation, with its invalid premise.
	23	Sections 2.7–2.8: source gain and model-scaling interpretation.
	24	Sections 2.7–2.8: reparameterization flaw; valid final gain.
	25	Section 2.7: all expanded and error-based final dynamics.
	26	Section 2.9: full learning/control update and warm-up assumptions.
	27	Section 2.9 and Figure 2.3: revised intelligent-system boundary.
	28	Closing acknowledgement only; no new academic content.
2000-ct-systems	1	Section 3 opening: title, subtitle, lecturer attribution, date.
	2	Section 3.1: nonlinear and LTI models, variables and dimensions.
	3	Section 3.2: quadrotor state, inputs, four motion equations.
	4	Section 3.3: manipulator model and first-order conversion.
	5	Section 3.4: equilibrium, stability, eigenvalues, controllability.
	6	Section 3.5: every row and column of the taxonomy.

File	Page	Content destination
3000-dt-systems	1	Section 4 opening and reading guide: title, subtitle, date, attribution.
	2	Section 4.1: nonlinear and LTI difference equations.
	3	Section 4.1: sampling, digital motivation, recursive predictions.
	4	Section 4.2: motor, Euler derivation and matrices.
	5	Sections 4.3–4.4: ZOH velocity, both integrals and Taylor expansion.
	6	Section 4.4: input matrices, within-step displacement, corrections.
	7	Section 4.5: Leslie state, matrix, fecundity, survival.
	8	Section 4.5: births, aging, three eigenvalue cases and assumptions.
	9	Section 4.6: random surfer, row-stochastic matrix, transpose.
	10	Section 4.6: damping 0.85/0.15, stationary distribution, iteration.
	11	Section 4.7: all SIR equations, parameters, reproduction condition.
	12	Section 4.8: Schur criterion and exact eigenvalue mapping.
	13	Section 4.9: Lyapunov equation and energy decrement.
	14	Section 4.10: full taxonomy and limits of its criteria.
1000-opt-ctr-ct-sys	1	Section 5 opening: title, subtitle, lecturer, date.
	2	Section 5.1: objective, dynamics, inputs, terminal set and time.
	3	Section 5.2: Hamiltonian, state, adjoint, minimization, transversality.
	4	Section 5.3: value function, HJB, boundary, feedback.
	5	Section 5.4: LQR cost, stationarity, ansatz, Riccati, gain.
	6	Section 5.5: all five comparison dimensions, with qualifications.
2000-opt-ctr-dt-sys	1	Section 6 opening: title, subtitle, lecturer, date.
	2	Section 6.1: horizon, objective, dynamics, inputs, terminal set.
	3	Section 6.2: Hamiltonian and four necessary conditions.
	4	Section 6.3: value function, Bellman equation, boundary, feedback.
	5	Section 6.4: DLQR cost, gain, Riccati, terminal matrix.
	6	Section 6.5: all six continuous/discrete comparison dimensions.
3000-RL4LLM	1	Section 7 opening: full title, subtitle, lecturer, overview, date.
	2	Section 7 opening and 7.10: all four agenda themes.
	3	Section 7.1: pretraining, SFT role, original prompt and target fragment.
	4	Section 7.1: autoregressive loss, masking, scale and data sources.
	5	Section 7.2: Alpaca, Vicuna, LIMA and the hypothesis qualification.
	6	Section 7.3: multiple answers, sequence rewards, exploration.
	7	Section 7.4: three RLHF stages, Bradley–Terry loss, KL objective.
	8	Section 7.5: DPO loss, implicit reward, all adoption examples.
	9	Section 7.6: R1-Zero, all group statistics, clipped surrogate, behaviour.
	10	Section 7.7: all four R1 stages, readability and language mixing.
	11	Section 7.8: all six comparison rows, with explicit qualifications.
	12	Section 7.9 and Figure 7.1: pipeline, examples, SFT and RL roles.
	13	Section 7.10 and Appendix C.3: FLAN, LIMA, InstructGPT, DPO.
	14	Section 7.10 and Appendix C.3: DeepSeekMath, V3, R1.
	15	Section 7.10 and Appendix C.3: both textbooks and reading scope.
Bertsekas, Ch. 1	2	Section 8 opening and 8.1: exact DP motivation; deterministic model, horizon, state and control.
	3	Section 8.1: stage/terminal costs, total cost, optimal value and deterministic stage diagram.
	4	Section 8.1: finite-state transition graph and shortest-path interpretation.

File	Page	Content destination
	5	Section 8.1: scheduling formulation, precedence constraints, state choice and setup costs.
	6	Section 8.1: scheduling interpretation and linear–quadratic oven example.
	7	Sections 8.1–8.2: oven constraints/optimization context and principle of optimality.
	8	Section 8.2: principle-of-optimality interpretation and backward tail-subproblem construction.
	9	Section 8.1: scheduling tail subproblems of length two and their cost-to-go values.
	10	Section 8.1: longer scheduling tails, original optimum 10, and optimal schedule $CABD$.
	11	Section 8.2: deterministic terminal condition, Bellman recursion, tail cost and induction step.
	12	Sections 8.2–8.3: forward reconstruction of an optimal sequence and start of value-space approximation.
	13	Section 8.3: approximate value-space control, approximate Q-factor and Q terminology.
	14	Sections 8.3–8.4: exact Q-factor recursion and stochastic dynamics/policy introduction.
	15	Section 8.4: stochastic feedback policies, random stage costs, expected policy cost.
	16	Section 8.4: stochastic Bellman recursion, optimal policy and four shared DP characteristics.
	17	Section 8.4: stochastic Q-factors, Q-only recursion and simulation point; Section 1.3 excluded.
Cellier–Kofman, Ch. 2	25–26	Section 9: book 2.1, initial-value model, step size, and Taylor approximation.
	26–31	Book 2.2: method order, local/global error, truncation, roundoff, accumulation, precision trade-offs, validation and verification.
	31–33	Book 2.3: explicit FE, implicit BE, and a simple worked example.
	34–42	Book 2.4: stability domains, step restrictions, stiffness, false stability, oscillations, and numerical boundary construction; Section 2.5 excluded.

C References and compilation

C.1 Primary lecture material

All academic topics are grounded in the following supplied PDFs:

- `1000-intro.pdf`
- `1500-video-compressor.pdf`
- `2000-ct-systems.pdf`
- `3000-dt-systems.pdf`
- `1000-opt-ctr-ct-sys.pdf`
- `2000-opt-ctr-dt-sys.pdf`
- `3000-RL4LLM.pdf`

The supplied *Biometric Systems* PDF was used only as a visual reference.

C.2 Assigned textbook excerpt

Dimitri P. Bertsekas. *Reinforcement Learning and Optimal Control*. Draft textbook, Athena Scientific, February 13, 2019. Section 8 uses Chapter 1, printed pp. 2–17 (PDF pp. 15–30): deterministic dynamic programming, the principle of optimality, approximation in value space, Q-factors, and stochastic dynamic programming. The notes stop before Section 1.3, *Examples, Variations, and Simplifications*, as requested.

François E. Cellier and Ernesto Kofman. *Continuous System Simulation*. Springer, 2006. Section 9 summarizes Chapter 2, Sections 2.1–2.4, printed pp. 25–42, stopping before Section 2.5. The summary uses simplified explanations and illustrative calculations rather than reproducing the book's full derivations or program listings. It distinguishes asymptotic stability from boundedness and states the smoothness qualifications for Taylor approximation.

C.3 Reference cited by the lecture

Antonio Filieri, Henry Hoffmann, and Martina Maggio. *Automated design of self-adaptive software with control-theoretical formal guarantees*. Proceedings of the 36th International Conference on Software Engineering (ICSE 2014), ACM, New York, NY, USA, pp. 299–310. doi:10.1145/2568225.2568272.

C.4 Literature cited by the LLM lecture

The following seven papers and two books reproduce the complete reading list on source pages 13–15. Dates refer to the versions or publication years cited by the lecture.

1. Jason Wei et al. (2021). *Finetuned Language Models Are Zero-Shot Learners*. ICLR 2022. FLAN and instruction tuning. arXiv:2109.01652.
2. Chunting Zhou et al. (2023). *LIMA: Less Is More for Alignment*. NeurIPS 2023. Data-efficient SFT and the alignment hypothesis. arXiv:2305.11206.
3. Long Ouyang et al. (2022). *Training language models to follow instructions with human feedback*. NeurIPS 2022. InstructGPT and the SFT–reward-model–PPO workflow. arXiv:2203.02155.
4. Rafael Rafailov et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023. Preference loss and implicit reward reparameterization. arXiv:2305.18290.

5. Zhihong Shao et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. Introduces GRPO. arXiv:2402.03300.
6. DeepSeek-AI (2024). *DeepSeek-V3 Technical Report*. Mixture-of-Experts (MoE) architecture and GRPO post-training. arXiv:2412.19437.
7. DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. R1-Zero and the hybrid R1 workflow. Technical checks here use the original report, version 1. arXiv:2501.12948v1.
8. Richard S. Sutton and Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. Second edition, MIT Press. Markov decision processes, value functions, and policy gradients.
9. Dan Jurafsky and James H. Martin (2024 draft, as cited). *Speech and Language Processing*. Third-edition draft; the slide recommends Chapters 10–13 for Transformers, fine-tuning, and alignment. Chapter numbering is draft-dependent: this is not a claim about every later revision.

C.5 Supplementary checks

The references below support targeted technical checks; they do not replace the supplied course material.

- Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. *Image quality assessment: From error visibility to structural similarity*. IEEE Transactions on Image Processing, vol. 13, no. 4, April 2004. Original paper: original SSIM paper. Used to qualify the lecture's operational SSIM range.
- Sanjay Lall, Stanford EE363. *Linear quadratic regulator: Discrete-time finite horizon*. Stanford EE363 notes. Used to cross-check the discrete Riccati recursion, gain, and definiteness assumptions.
- Stanford EE363. *Continuous time linear quadratic regulator*. Stanford EE363 notes. Used to cross-check the differential Riccati equation, feedback gain, and backward terminal-value formulation.

The compressor characteristic polynomial and motor discretization corrections follow directly from symbolic expansion and integration of the equations in the supplied slides. They do not depend on an alternate plant model.

Additional primary checks for Section 7. Historical counts were checked against Stanford CRFM's *Alpaca* announcement (Stanford CRFM Alpaca announcement) and LMSYS's *Vicuna* announcement (LMSYS Vicuna announcement). DPO attribution was checked against the Zephyr-7B-beta model card (Zephyr-7B-beta model card) and Dubey et al., *The Llama 3 Herd of Models* (2024), arXiv:2407.21783. The official Mixtral-8x22B-Instruct-v0.1 card was reviewed without establishing the lecture's specific DPO attribution (official Mixtral-8x22B-Instruct model card); Mistral explicitly documents DPO for Mixtral-8x7B in *Mixtral of experts* (Mistral, "Mixtral of experts"). The original DPO, LIMA, and R1 papers listed above support the corresponding mathematical and factual qualifications.

C.6 Compiling the source

Run `build.ps1` to compile with pdfLaTeX until cross-references stabilize. Keep the PNG cover in `assets`; all diagrams use TikZ. The source requires standard TeX Live/MiKTeX packages, including Palatino text and mathematics and `microtype`. The editor directive selects pdfLaTeX; VS Code's LaTeX Workshop requires an installed distribution.